2.3.5 Материалы для изучения по научной специальности Математическое и программное обеспечение вычислительных систем, комплексов и компьютерных сетей Гусев Виталий Евгеньевич 2.3.5 2026

Содержание

1 Раздел 1. Теория вычислительных систем **2**

1.1 Понятие алгоритма и его уточнения 2
1.2 Машины Тьюринга 5
1.3 Нормальные алгоритмы Маркова 7
1.4 Рекурсивные функции 13
1.5 Эквивалентность данных формальных моделей алгоритмов 15
1.6 Понятие об алгоритмической неразрешимости. Примеры алгоритмически неразрешимых проблем 16
1.7 Понятие сложности алгоритмов 21
1.8 Классы P и NP 22
1.9 Полиномиальная сводимость задач 25
1.10 Теорема Кука об NP-полноте задачи выполнимости булевой формулы . 27
1.11 Примеры NP-полных задач, подходы к их решению 31
1.12 Точные и приближенные комбинаторные алгоритмы 33
1.13 Примеры эффективных (полиномиальных) алгоритмов 35
1.14 Автоматы. Эксперименты с автоматами. Алгебры регулярных выражений. Теорема Клини о регулярных языках 42

1 Раздел 1. Теория вычислительных систем

1.1 Понятие алгоритма и его уточнения

Взято из 1.1. Алгоритмы в жизни и в математике

Алгоритмы в жизни. Понятие алгоритма стихийно формировалось с древнейших времён. Современный человек понимает под алгоритмом чёткую систему инструкций о выполнении в определённом порядке некоторых действий для решения всех задач какого-то данного класса.

Многочисленные и разнообразные алгоритмы окружают нас буквально во всех сферах жизни и деятельности. Многие наши действия доведены до бессознательного автоматизма, мы порой и не осознаём, что они регламентированы неким алгоритмом — чёткой системой инструкций. Например, наши действия при входе в магазин «Универсам» (сдать свою сумку, получить корзину с номером, пройти в торговый зал, заполнить корзину продуктами, оплатить покупку в кассе, предъявить чек контролёру, взять свою сумку, переложить в неё продукты, сдать корзину, покинуть магазин). Второй пример — приготовление манной каши (500 мл молока довести до кипения, при тщательном помешивании засыпать 100 г манной крупы, при помешивании довести до кипения и варить 10 минут). Автоматизм выполнения этих и многих других действий не позволяет нам осознавать их алгоритмическую сущность.

Но есть немало таких действий, выполняя которые мы тщательно получив рецепт (алгоритм) его приготовления, постараетесь выполнить в указанной последовательности все его предписания.

Алгоритмы в математике. Большое количество алгоритмов встречается при изучении математики буквально с первых классов школы. Это, прежде всего, алгоритмы выполнения четырёх арифметических действий над различными числами — натуральными, целыми, дробными, комплексными. Вот пример такого алгоритма: «Чтобы из одной десятичной дроби вычесть другую, надо: 1) уравнять число знаков после запятой в уменьшаемом и вычитаемом; 2) записать вычитаемое под уменьшаемым так, чтобы запятая оказалась под запятой; 3) произвести вычитание так, как вычитают натуральные числа; 4) поставить в полученной разности, запятую под запятыми в уменьшаемом и вычитаемом». Вот пример алгоритма сложения приближенных чисел. Найти сумму чисел p, q и s, где $p \approx 3,1416$, $q \approx 2,718$ и $s \approx 7,45$.

1. Выделим слагаемое с наименьшим числом десятичных знаков. Таким слагаемым является число 7,45 (два десятичных знака).

2. Округлим остальные слагаемые, оставляя в них столько десятичных знаков, сколько их имеется в выделенном слагаемом: $3,1416 \approx 3,14$; $2,718 \approx 2,72$.

3. Выполним сложение приближённых значений чисел: $3,14+2,72+7,45 = 13,31$.

Итак, $p + q + s \approx 13,31$.

Немало алгоритмов в геометрии: алгоритмы геометрических построений.

Взято из 1.2. Неформальное понятие алгоритма и необходимость его уточнения

Неформальное понятие алгоритма. Прежде чем перейти к математическому изучению понятия алгоритма, постараемся внимательно проанализировать рассмотренные примеры алгоритмов и выявить их общие типичные черты и особенности.

1. Каждый алгоритм предполагает наличие некоторых начальных или исходных данных, а в результате применения приводит к получению определённого искомого результата. Например, в алгоритме с проявителем начальные данные — содержимое большого и малого пакетов, вода. Искомый результат — готовый к употреблению проявитель для плёнки. При вычислении ранга матрицы начальными данными служит прямоугольная таблица, составленная из $m \times n$ рациональных чисел, результат — натуральное число, являющееся рангом данной матрицы.

   Говоря о начальных данных для алгоритма, имеют в виду так называемые допустимые начальные данные, т.е. такие начальные данные, которые сформулированы в терминах данного алгоритма. Так, к числу допустимых начальных данных для алгоритма варки манной каши никак не отнесёшь элементы множества натуральных чисел, а к числу начальных данных алгоритма Евклида — молоко и манную крупу (или даже комплексные числа).

   Среди допустимых начальных данных для алгоритма могут быть такие, к которым он применим, т.е. отправляясь от которых можно получить искомый результат, а могут быть такие, к которым данный алгоритм неприменим, т.е. отправляясь от которых искомого результата получить нельзя. Неприменимость алгоритма к допустимым начальным данным может заключаться либо в том, что алгоритмический процесс никогда не оканчивается (в этом случае говорят, что он бесконечен), либо в том, что его выполнение во время одного из шагов наталкивается на препятствие, заходит в тупик (в этом случае говорят, что он безрезультатно обрывается). Проиллюстрируем на примерах оба случая.

   Пример 1. Приведём пример бесконечного алгоритмического процесса. Всем известен алгоритм деления десятичных дробей. Числа 5,1 и 3 являются для него допустимыми начальными данными, применение к которым алгоритма деления приводит к искомому результату 1,7. Иная картина наблюдается, если разделить 5,1 на 7. Процесс будет бесконечным, так как деление не завершится.

   Пример 2. Теперь приведём пример алгоритма, заходящего в тупик, безрезультатно обрывающегося. Вот его предписания:

   (a) Исходное число умножить на 2. Перейти к выполнению п. 2;

   (b) К полученному числу прибавить 1. Перейти к выполнению п. 3;

   (c) Определить остаток y от деления полученной в п. 2 суммы на 3. Перейти к выполнению п. 4;

   (d) Разделить исходное число на y. Частное является искомым результатом. Конец.

   Пусть натуральные (целые положительные) числа будут допустимыми начальными данными для этого алгоритма. Для числа 6 алгоритмический процесс будет проходить так: Первый шаг: $6 \cdot 2 = 12$; переходим к п. 2;
   Второй шаг: $12 + 1 = 13$; переходим к п. 3;
   Третий шаг: $y = 1$, переходим к п. 4;
   Четвёртый шаг: $6 : 1 = 6$. Конец.

   Искомый результат равен 6. Иначе будет протекать алгоритмический процесс для исходного данного 7: Первый шаг: $7 \cdot 2 = 14$; переходим к п. 2;

Второй шаг: $14 + 1 = 15$; переходим к п. 3;
Третий шаг: $y = 0$; переходим к п. 4;
Четвёртый шаг: $7 : 0$ — деление невозможно. Процесс зашёл в тупик, натолкнулся на препятствие и безрезультатно оборвался.

Наконец отметим ещё одно немаловажное качество начальных данных для алгоритма. Они должны быть конструктивными. Конструктивность объекта означает, что он весь целиком может быть представлен для его рассмотрения и работы с ним. Примеры: булевы функции; формулы алгебры высказываний; натуральные числа; рациональные числа; матрицы с натуральными и рациональными элементами; многочлены с рациональными коэффициентами. Неконструктивные объекты: действительные числа, представимые бесконечными непериодическими дробями $0, \alpha_1\alpha_2\alpha_3\ldots$, в частности числа π и e.

2. Далее, применение каждого алгоритма осуществляется путём выполнения дискретной цепочки, последовательности неких элементарных действий. Эти действия называют шагами или тактами, а процесс их выполнения называют алгоритмическим процессом. Причём, начальными данными для каждого последующего шага являются данные, полученные в результате выполнения предыдущего шага. Шаги должны быть достаточно элементарными и заканчиваться результатом (то есть быть эффективными). Таким образом, отмечается свойство **дискретности** алгоритма, которое означает, что к результату мы приходим, выполняя определенные действия пошаговым, дискретным образом.

3. Непременным условием, которому удовлетворяет алгоритм, является его **детерминированность** или **определённость**. Это означает, что предписания алгоритма с равным успехом могут быть выполнены любым другим человеком и в любое другое время, причём результат получится тот же самый. Другими словами, предписания алгоритма настолько точны, отчётливы, что не допускают никаких двусмысленных толкований и никакого произвола со стороны исполнителя. Они единственным и вполне определённым путём всякий раз приводят к искомому результату. Это наводит на мысль, что выполнение тех или иных алгоритмов может быть поручено машине, что широко и делается на практике.

4. Существенной чертой алгоритма является его **массовый характер**, т.е. возможность применять его к обширному классу начальных данных, возможность достаточно широко эти начальные данные варьировать. Другими словами, каждый алгоритм призван решить ту или иную массовую проблему, т.е. решать класс однотипных задач. Например, задача нахождения наибольшего общего делителя чисел 4 и 6 есть единичная проблема (можно решить её и без применения алгоритма Евклида), но задача нахождения наибольшего общего делителя произвольных натуральных чисел m и n — уже проблема массовая. Суть алгоритма Евклида состоит в том, что он приводит к желаемому результату вне зависимости от выбора конкретной пары натуральных чисел, в то время как при решении указанной единичной проблемы можно предложить такой способ, который окажется неприменимым для другой пары натуральных чисел.

Итак, подводя итоги обсуждению характерных свойств и особенностей алгоритма, можем сформулировать следующее интуитивно описательное определение этого понятия.

Под **алгоритмом** понимается чёткая система инструкций, определяющая дискретный детерминированный процесс, ведущий от варьируемых начальных данных (входов) к искомому результату (выходу), если таковой существует, через конечное число тактов работы алгоритма; если же искомого результата не существует, то вычислительный процесс либо никогда не оканчивается, либо попадает в тупик.

Отметим в заключение, что сам термин «алгоритм» (или «алгорифм») происходит от имени великого среднеазиатского учёного Мухаммеда аль-Хорезми (787 – ок. 850). В своём трактате, написанном по-арабски, латинская версия которого относится к XII веку и начинается словами «Dixit algorizm», то есть «Сказал аль-Хорезми», им среди прочего была описана индийская позиционная система чисел и сформулированы правила выполнения четырёх арифметических действий над числами в десятичной записи.

Необходимость уточнения понятия алгоритма. Понятие алгоритма формировалось с древнейших времён, но до конца первой трети XX века математики довольствовались интуитивным представлением об этом объекте. Термин алгоритм употреблялся в математике лишь в связи с теми или иными конкретными алгоритмами. Утверждение о существовании алгоритма для решения задач данного типа сопровождалось фактическим его описанием.

Парадоксы, обнаруженные в основаниях математики в начале XX века, вызвали к жизни различные концепции и течения, призванные эти парадоксы устранить. В 20-е годы вплотную встали вопросы о том, что же такое строгая выводимость и эффективное вычисление. Понятие алгоритма само должно было стать объектом математического исследования и поэтому нуждалось в строгом определении. Кроме того, к этому вынуждало развитие физики и техники, быстро приближавшее начало века электронно-вычислительных машин.

Далее, в начале XX века у математиков начали возникать подозрения в том, что некоторые массовые задачи по-видимому не имеют алгоритмического решения. Для точного доказательства несуществования какого-то объекта необходимо иметь его точное математическое определение. Совершенно аналогичная ситуация сложилась в своё время в математике, когда назрела необходимость уточнения таких понятий, как непрерывность, кривая, поверхность, длина, площадь, объём и т.п.

Первые работы по уточнению понятия алгоритма и его изучению, т.е. по теории алгоритмов, были выполнены в 1936-1937 годах математиками А. Тьюрингом, Э. Постом, Ж. Эрбраном, К. Гёделем, А.А. Марковым, А. Чёрчем. Было выработано несколько определений понятия алгоритма, но впоследствии выяснилось, что все они равносильны между собой, т.е. определяют одно и то же понятие.

1.2 Машины Тьюринга

Взято из 2.1. Понятие машины Тьюринга и применение машин Тьюринга к словам

Определение машины Тьюринга. Машина Тьюринга есть математическая (воображаемая) машина, а не машина физическая. Она есть такой же математический объект, как функция, производная, интеграл, группа и т.д. И так же, как и другие математические понятия, понятие машины Тьюринга отражает объективную реальность, моделирует некие реальные процессы. Именно, Тьюринг предпринял попытку смоделировать действия математика (или другого человека), осуществляющего некую умственную созидательную деятельность. Такой человек, находясь в опре-

делённом «умонастроении» («состоянии»), просматривает некоторый текст. Затем он вносит в этот текст какие-то изменения, проникается новым «умонастроением» и переходит к просмотру последующих записей.

Машина Тьюринга действует примерно так же. Её удобно представлять в виде автоматически работающего устройства. В каждый дискретный момент времени устройство, находясь в некотором состоянии, обозревает содержимое одной ячейки протягиваемой через устройство ленты и делает шаг, заключающийся в том, что устройство переходит в новое состояние, изменяет содержимое обозреваемой ячейки и переходит к обозрению следующей ячейки, справа или слева. Причём шаг осуществляется на основании предписанной команды. Совокупность всех команд представляет собой программу машины Тьюринга.

Опишем теперь машину Тьюринга θ более тщательно. Машина θ располагает конечным числом знаков (символов, букв), образующих так называемый внешний алфавит $A = \{a_0, a_1, \ldots, a_n\}$. В каждую ячейку обозреваемой ленты в каждый дискретный момент времени может быть записан только один символ из алфавита A. Ради единообразия удобно считать, что среди букв внешнего алфавита A имеется «пустая буква» и именно она записана в пустую ячейку ленты. Условимся, что «пустой буквой» или символом пустой ячейки является буква a_0. Лента предполагается неограниченной в обе стороны, но в каждый момент времени на ней записано конечное число непустых букв.

Далее, в каждый момент времени машина θ способна находиться в одном состоянии из конечного числа внутренних состояний, совокупность которых $Q = \{q_0, q_1, \ldots, q_m\}$. Среди состояний выделяются два — начальное q_1 и заключительное, или состояние остановки, q_0. Находясь в состоянии q_1, машина начинает работать. Попав в состояние q_0, машина останавливается.

Работа машины θ определяется программой (функциональной схемой). Программа состоит из команд. Каждая команда $T(i, j)$ $(i = 1, 2, \ldots, m; j = 0, 1, \ldots, n)$ представляет собой выражение одного из следующих видов:

$$q_i a_j \to q_k a_l C, \quad q_i a_j \to q_k a_l \text{П}, \quad q_i a_j \to q_k a_l \text{Л},$$

где $0 \le k \le m$, $0 \le l \le n$. В выражениях первого вида символ C будем часто опускать.

Как же работает машина Тьюринга? Находясь в какой-либо момент времени в незаключительном состоянии (т.е. в состоянии отличном от q_0), машина совершает шаг, который полностью определяется ее текущим состоянием q_i и символом a_j, воспринимаемым ею в данный момент на ленте. При этом содержание шага регламентировано соответствующей командой $T_{i,j} : q_i a_j \to q_k a_l X$, где $X \in \{C, \text{П}, \text{Л}\}$. Шаг заключается в том, что:

1. содержимое a_j обозреваемой на ленте ячейки стирается, и на его место записывается символ a_l (который может совпадать с a_j);

2. машина переходит в новое состояние q_k (оно также может совпадать с предыдущим состоянием q_i);

3. машина переходит к обозрению следующей правой ячейки от той, которая обозревалась только что, если $X = \text{П}$, или к обозрению следующей левой ячейки, если $X = \text{Л}$, или же продолжает обозревать ту же ячейку ленты, если $X = C$.

В следующий момент времени (если $q_k \neq q_0$) машина делает шаг, регламентированный командой $T_{k,l} : q_k a_l \to q_r a_s X$, и т.д.

Поскольку работа машины по условию полностью определяется её состоянием q_i в данный момент и содержимым a_j обозреваемой в этот момент ячейки, то для каждых q_i и a_j $(i = 1, 2, \ldots, m; j = 0, 1, \ldots, n)$ программа машины должна содержать одну и только одну команду, начинающуюся символами $q_i a_j$. Поэтому программа машины Тьюринга с внешним алфавитом $A = \{a_0, a_1, \ldots, a_n\}$ и алфавитом внутренних состояний $Q = \{q_0, q_1, \ldots, q_m\}$ содержит $m(n+1)$ команд.

Словом в алфавите A или в алфавите Q, или в алфавите $A \cup Q$ называется любая последовательность букв соответствующего алфавита. Под k-ой **конфигурацией** будем понимать изображение ленты машины с информацией, сложившейся на ней к началу k-го шага (или слово в алфавите A, записанное на ленту к началу k-го шага), с указанием того, какая ячейка обозревается в этот шаг и в каком состоянии находится машина. Имеют смысл лишь конечные конфигурации, т.е. такие, в которых все ячейки ленты, за исключением, быть может, конечного числа, пусты. Конфигурация называется **заключительной**, если состояние, в котором при этом находится машина, заключительное.

Если выбрать какую-либо незаключительную конфигурацию машины Тьюринга в качестве исходной, то работа машины будет состоять в том, чтобы последовательно, шаг за шагом, преобразовывать исходную конфигурацию в соответствии с программой машины до тех пор, пока не будет достигнута заключительная конфигурация. После этого работа машины Тьюринга считается закончившейся, а результатом работы считается достигнутая заключительная конфигурация.

Будем говорить, что непустое слово α в алфавите $A \setminus \{a_0\} = \{a_1, \ldots, a_n\}$ воспринимается машиной в **стандартном положении**, если оно записано в последовательных ячейках ленты, все другие ячейки пусты, и машина обозревает крайнюю справа ячейку из тех, в которых записано слово α. Стандартное положение называется **начальным** (**заключительным**), если машина, воспринимающая слово в стандартном положении, находится в начальном состоянии q_1 (соответственно, в состоянии остановки q_0). Наконец, будем говорить, что слово α **перерабатывается** машиной в слово β, если от слова α, воспринимаемого в начальном стандартном положении, машина после выполнения конечного числа команд приходит к слову β, воспринимаемому в положении остановки.

1.3 Нормальные алгоритмы Маркова

Взято из 4.1. Марковские подстановки, нормальные алгоритмы и нормально вычислимые функции

Марковские подстановки. Алфавитом (как и прежде) называется любое непустое множество. Его элементы называются **буквами**, а любые последовательности букв — **словами** в данном алфавите. Для удобства рассуждений допускаются пустые слова (они не имеют в своем составе ни одной буквы). Пустое слово будем обозначать Λ. Если A и B — два алфавита, причём $A \subseteq B$, то алфавит B называется расширением алфавита A.

Слова будем обозначать латинскими буквами: P, Q, R, S, T, U, V, W (или этими же буквами с индексами). Слово T называется **составной частью** другого слова V, или **подсловом** второго, если $V = PTQ$ для некоторых слов P и Q.

Работа нормального алгоритма A, задаваемого приведённой выше схемой, может

быть описана следующим образом. Пусть дано слово V в алфавите A. Ищем первую в схеме алгоритма A формулу подстановки $P_k \to (\cdot)Q_k$ такую, что слово P_k входит в слово V. Если ни одно из слов $P_1, \ldots, P_r$ не входит в V, то алгоритм A не применим к слову V. Если находим первую в схеме алгоритма A формулу подстановки $P_k \to (\cdot)Q_k$ такую, что слово P_k входит в слово V, то совершаем подстановку слова Q_k вместо самого левого (первого) вхождения слова P_k в слово V. Пусть V_1 — результат такой подстановки. Если $P_k \to (\cdot)Q_k$ — формула заключительной подстановки, то работа алгоритма заканчивается, и его результатом является слово V_1: $A(V) = V_1$. Если подстановка $P_k \to (\cdot)Q_k$ не является заключительной, то применяем к слову V_1 тот же поиск, который был только что применён к слову V, и так далее. Если мы в конце концов получим такое слово V_m $(m \geq 1)$, что алгоритм A не применим к V_m (т.е. ни одно из слов $P_1, \ldots, P_r$ не входит в V_m), то работа алгоритма A заканчивается и V_m будет его результатом: $A(V) = V_m$. При этом, возможно, что описанный процесс никогда не закончится (будет продолжаться бесконечно). В таком случае также будем считать, что алгоритм A не применим к слову V.

Мы определили понятие нормального алгоритма в алфавите A. Если же алгоритм задан в некотором расширении алфавита A, то говорят, что он есть нормальный алгоритм над A.

Различие между понятием нормального алгоритма в алфавите A и понятием нормального алгоритма над алфавитом A является весьма существенным и важным. Всякий нормальный алгоритм в алфавите A использует только буквы из A, тогда как нормальный алгоритм над алфавитом A может использовать и некоторые дополнительные буквы, не входящие в A, хотя применяться он будет к словам в алфавите A и выдавать будет в результате слово в алфавите A. Ясно, что всякий нормальный алгоритм в алфавите A будет также и нормальным алгоритмом над алфавитом A. В то же время обратное утверждение не верно: можно показать, что существуют, вообще говоря, алгоритмы в A, которые определяются нормальными алгоритмами над A, но сами не являются нормальными алгоритмами в A.

Рассмотрим примеры нормальных алгоритмов.

ПРИМЕР 4.1.4. Пусть $A = \{a, b\}$ — алфавит. Рассмотрим следующую схему нормального алгоритма в алфавите B (над алфавитом A):

$$\begin{cases} \alpha\alpha \to \beta & (a) \\ \beta\xi \to \xi\beta \quad (\xi \in A) & (b) \\ \beta\alpha \to \beta & (c) \\ \beta \to \cdot\Lambda & (d) \\ \alpha\eta\xi \to \xi\alpha\eta \quad (\xi, \eta \in A) & (e) \\ \Lambda \to \alpha & (f) \end{cases}$$

Здесь в подстановках (b) и (e) используются переменные ξ, η, вместо которых могут подставляться любые буквы из алфавита A. Проверьте, что данный алгоритм следующим образом осуществляет переработку, например, слова $a_1a_2a_3$:

$$a_1a_2a_3 \xrightarrow{f} \alpha a_1a_2a_3 \xrightarrow{e} a_2\alpha a_1a_3 \xrightarrow{e} a_2a_3\alpha a_1 \xrightarrow{f} \alpha a_2a_3\alpha a_1 \xrightarrow{e} a_3\alpha a_2\alpha a_1 \xrightarrow{f} \ldots$$

Из работы алгоритма на этом примере ясно, что он любое непустое (любой длины) слово V в алфавите A перерабатывает в его отражение V'.

ПРИМЕР 4.1.7. Пусть алфавит A не содержит букв α, β, γ, и пусть $B = A \cup \{\alpha\}$ и $C = A \cup \{\alpha, \beta, \gamma\}$.

а) Постройте нормальный алгоритм A в алфавите B (над алфавитом A), который в каждом непустом слове в алфавите A стирал бы первую букву, а пустое слово переводил бы в пустое, т.е. $A(\Lambda) = \Lambda$ и $A(aV) = V$ для любой буквы $a \in A$ и любого слова V в A.

б) Постройте нормальный алгоритм A в алфавите C (над алфавитом A), который удваивал бы любое слово в алфавите A, т.е. любое слово V в алфавите A переводил бы в слово VV: $A(V) = VV$.

Рассмотрим ещё два примера нормальных алгоритмов в алфавите A.

ПРИМЕР 4.1.8. В алфавите $A = \{1\}$ схема, состоящая из одной подстановки $\Lambda \to \cdot 1$, определяет нормальный алгоритм, который к каждому слову в алфавите $A = \{1\}$ (все такие слова суть следующие: $\Lambda, 1, 11, 111, 1111, 11111$ и т. д.) приписывает слева 1.

ПРИМЕР 4.1.9. В алфавите $A = \{1\}$ построить нормальный алгоритм, который для всякого слова V в этом алфавите выдавал бы слово 1, если число единиц в слове V делится на 3, и выдавал бы пустое слово Λ в противном случае.

Рассмотрим нормальный алгоритм в алфавите $A = \{1\}$ со следующей схемой:

$$\begin{cases} 111 \to \Lambda & (a) \\ 11 \to \cdot\Lambda & (b) \\ 1 \to \cdot\Lambda & (c) \\ \Lambda \to \cdot 1 & (d) \end{cases}$$

Этот алгоритм работает следующим образом: пока число букв 1 в слове не меньше 3, алгоритм последовательно стирает по три буквы (подстановка (a)). Если при этом все буквы оказались стёртыми, то подстановка (d) выдаёт 1 и заканчивает работу алгоритма. Если оказались не стёртыми две единицы 11, то подстановка (b) заменяет их на пустое слово и заканчивает работу алгоритма. Наконец, если не стёртой оказалась одна единица 1, то подстановка (c) заменяет её на пустое слово и заканчивает работу алгоритма. Например:

$$111111 \xrightarrow{a} 111 \xrightarrow{a} 1 \xrightarrow{c} \Lambda;$$

$$11111111 \xrightarrow{a} 11111 \xrightarrow{a} 11 \xrightarrow{b} \Lambda \xrightarrow{d} 1.$$

В двух последних примерах 4.1.8 и 4.1.9 мы фактически рассмотрели нормальные алгоритмы, вычисляющие некоторые функции, а именно, в первом примере функцию следования $S(n) = n + 1$, а во втором — функцию $\varphi_3(n)$, определяемую следующим образом:

$$\varphi_3(11\ldots1) = \begin{cases} 1, & \text{если } n \text{ делится на 3;} \\ \Lambda, & \text{если } n \text{ не делится на 3;} \end{cases}$$

где n — число единиц в слове $11\ldots1$.

Как и машины Тьюринга, нормальные алгоритмы не производят собственно вычислений: они лишь производят преобразования слов, заменяя в них одни буквы другими, по предписанным им правилам. В свою очередь, мы предписываем им такие правила, результаты применения которых мы уже можем интерпретировать как вычисления. Рассмотренные два примера наглядно демонстрируют это.

Следующий пример в этом отношении будет ещё более ярким. В нём мы организуем процесс вычисления с помощью нормального алгоритма функции следования

$S(n) = n + 1$, не в единичной системе счисления (как это сделано в примере 4.1.8), а в привычной человеку десятичной системе счисления, хотя никаких привычных нам арифметических действий по сложению чисел при этом выполняться не будет.

ПРИМЕР 4.1.10. Построим нормальный алгоритм для вычисления функции $S(n) = n+1$ в десятичной системе счисления. В качестве алфавита возьмём перечень арабских цифр $A = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$, а нормальный алгоритм будем строить в его расширении $B = A \cup \{a, b\}$. Вот схема этого нормального алгоритма (читается по столбцам):

$$
\begin{array}{lll}
0b \to \cdot 1 & a0 \to 0a & 0a \to 0b \\
1b \to \cdot 2 & a1 \to 1a & 1a \to 1b \\
2b \to \cdot 3 & a2 \to 2a & 2a \to 2b \\
3b \to \cdot 4 & a3 \to 3a & 3a \to 3b \\
4b \to \cdot 5 & a4 \to 4a & 4a \to 4b \\
5b \to \cdot 6 & a5 \to 5a & 5a \to 5b \\
6b \to \cdot 7 & a6 \to 6a & 6a \to 6b \\
7b \to \cdot 8 & a7 \to 7a & 7a \to 7b \\
8b \to \cdot 9 & a8 \to 8a & 8a \to 8b \\
9b \to b0 & a9 \to 9a & 9a \to 9b \\
b \to \cdot 1 & & \Lambda \to a
\end{array}
$$

Попытаемся применить алгоритм к пустому слову Λ. Нетрудно понять, что на каждом шаге должна будет применяться самая последняя формула данной схемы. Получается бесконечный процесс:

$$\Lambda \Longrightarrow a \Longrightarrow aa \Longrightarrow aaa \Longrightarrow aaaa \Longrightarrow \ldots$$

Это означает, что к пустому слову данный алгоритм неприменим.

Если применить теперь алгоритм к слову 499, получим следующую последовательность слов:

$$499 \Longrightarrow a499 \Longrightarrow 4a99 \Longrightarrow 49a9 \Longrightarrow 499b \Longrightarrow 49b0 \Longrightarrow 4b00 \Longrightarrow 500$$

Таким образом, слово 499 перерабатывается данным нормальным алгоритмом в слово 500. Проверьте самостоятельно, что $328 \Longrightarrow 329$, $789 \Longrightarrow 790$.

В рассмотренном примере нормальный алгоритм построен в алфавите B, являющемся существенным расширением алфавита A до есть $A \subseteq B$ и $A \neq B$, но данный алгоритм слова в алфавите A перерабатывает снова в слова в алфавите A. Так что, рассмотренный алгоритм задан над алфавитом A.

Наша задача теперь будет состоять в том, чтобы охарактеризовать процесс вычисления с помощью нормальных алгоритмов функций от любого числа аргументов, заданных в множестве натуральных чисел и принимающих значения в этом же множестве. Такие функции мы будем называть нормально вычислимыми. Затем нужно будет как-то описать все такие функции, охарактеризовать их.

Нормально вычислимые функции. Итак, займёмся уточнением понятия вычислимости с помощью нормальных алгоритмов натуральных функций от натуральных аргументов.

Прежде всего, нужно условиться о том, в каком виде будут использоваться в нормальных алгоритмах натуральные числа. Для этого зафиксируем алфавит $A_1 = \{1\}$ и его расширение $A_2 = \{1, *\}$. Для всякого натурального числа n определим по индукции следующие слова в алфавите A_1: $\overline{0} = 1$, $\overline{n+1} = \overline{n}1$, т.е. $\overline{1} = 11$, $\overline{2} = 111$ и т.д.

Слова $\overline{n}$ будем называть числами. Таким образом, натуральное число n мы кодируем словом в алфавите A_1, состоящим из $n+1$ единицы. Будем также обозначать слово $11\ldots1$, состоящее из n единиц, символом 1^n. Тогда наше кодирование выглядит следующим образом: $\overline{0}=1^1$, $\overline{1}=1^2$, $\overline{2}=1^3$, $\ldots$, $\overline{n}=1^{n+1}$, $\ldots$

Тогда для рассмотренного в примере 4.1.8 алгоритма A имеем: $A(\overline{n})=\overline{n+1}$ для любого натурального n.

Теперь договоримся о кодировании упорядоченных n-ок натуральных чисел. (Эта процедура понадобится нам для описания процесса нормальной вычислимости функций n натуральных аргументов). Для этого мы используем расширенный алфавит $A_2=\{1,*\}$. Поставим в соответствие каждой упорядоченной n-ке $(k_1,k_2,\ldots,k_n)$, где $k_1,k_2,\ldots,k_n$ — натуральные числа, слово $\overline{k_1}*\overline{k_2}*\cdots*\overline{k_n}$ в алфавите A_2, которое обозначим через $\overline{(k_1,k_2,\ldots,k_n)}$. Так, например, $\overline{(1,4,0,2)}$ обозначает слово $11*11111*1*111$. Итак,

$$\overline{(k_1,k_2,\ldots,k_n)}=\overline{k_1}*\overline{k_2}*\cdots*\overline{k_n}.$$

Пусть теперь $f(x_1,x_2,\ldots,x_n)$ — частичная функция n аргументов, заданная на множестве N натуральных чисел и принимающая значения в том же множестве, т.е. $f:N^n\to N$.

ОПРЕДЕЛЕНИЕ 4.1.11. Будем называть частичную функцию $f(x_1,x_2,\ldots,x_n)$ **частично вычислимой по Маркову** (или частично нормально вычислимой), если существует нормальный алгоритм, вычисляющий эту функцию, т.е. такой нормальный алгоритм A_f, над алфавитом $A_2=\{1,*\}$, что

$$A_f(\overline{(k_1,k_2,\ldots,k_n)})=\overline{f(k_1,k_2,\ldots,k_n)}$$

всякий раз, когда хотя бы одна из частей этого равенства определена (тогда определена и другая часть этого равенства и эти части равны). При этом предполагается, что нормальный алгоритм A_f не применим к словам, отличным от слов вида $\overline{(k_1,k_2,\ldots,k_n)}$. Если функция $f(x_1,x_2,\ldots,x_n)$ определена всюду, т.е. для любой упорядоченной n-ки натуральных чисел, и является частично вычислимой по Маркову, то будем называть её **вычислимой по Маркову** (или нормально вычислимой).

Рассмотрим примеры нормально вычислимых функций.

ПРИМЕР 4.1.12. Построим нормальный алгоритм A_0 над алфавитом $A_2=\{1,*\}$, вычисляющий нуль-функцию $O(x)=0$ для любого $x\in\mathbb{N}$. Он должен быть применим к словам вида $\overline{n}=1^{n+1}$ и только к таким словам, причём, на каждом таком слове он должен выдавать результат $\overline{0}=1$, т.е. $A_0(\overline{n})=\overline{0}$ для любого слова $\overline{n}$. Покажем, что такой алгоритм может быть задан следующей схемой в расширенном алфавите $B=\{1,*,\alpha\}\supset A_2$:

$$\begin{cases} *\to * & (a)\\ \alpha 11\to\alpha 1 & (b)\\ \alpha 1\to\cdot 1 & (c)\\ \Lambda\to\alpha & (d)\end{cases}$$

В самом деле, проанализируем работу этого нормального алгоритма. Во-первых, в данном слове происходит поиск вхождения буквы $*$. Если такое вхождение находится (а это будет означать, что данное слово не есть число, т.е. не имеет вид $\overline{n}=1^{n+1}$), тогда выполняется первая подстановка схемы $*\to *$, затем снова будет выполняться эта подстановка, затем снова, и т.д., то есть алгоритм зацикливается, работает вечно, и это означает, что данный алгоритм не применим к словам, содержащим букву $*$, т.е. к словам, не имеющим вид $\overline{n}$.

Если вхождения буквы $*$ в данное слово не обнаружено, то происходит поиск вхождения слова $\alpha 11$; его нет, так как слово составлено из букв алфавита $A_2 = \{1, *\}$ ($\alpha \notin A_2$). Затем происходит поиск вхождения слова $\alpha 1$; его также нет по той же причине. Наконец, отрабатывается подстановка $\Lambda \to \alpha$. Она даёт результат, ибо всякое слово V можно себе мыслить как начинающееся с пустого слова — ΛV. Эта подстановка приписывает в начало данного слова букву α. Дальше вступает в действие подстановка (b), стирая при каждом её применении одну единицу, пока не останется всего одна единица. Тогда заключительная подстановка (c) завершает работу алгоритма, стирая букву α и оставляя результатом единственную единицу, которая служит кодировкой натурального числа 0. Например:

$$11111 \xrightarrow{d} \alpha 11111 \xrightarrow{b} \alpha 1111 \xrightarrow{b} \alpha 111 \xrightarrow{b} \ldots \xrightarrow{b} \alpha 11 \xrightarrow{b} \alpha 1 \xrightarrow{c} 1.$$

ПРИМЕР 4.1.13. Построить нормальный алгоритм A_1 над алфавитом $A_2 = \{1, *\}$, вычисляющий функцию следования $S(x) = x + 1$ для любого $x \in \mathbb{N}$. Он также должен быть применим к словам вида $\overline{n} = 1^{n+1}$ и только к таким словам, причём, на каждом таком слове он должен выдавать результат $\overline{n+1} = 1^{n+2}$, т.е. $A_1(\overline{n}) = \overline{n+1}$ для любого слова $\overline{n}$. Нетрудно понять, что такой алгоритм может быть задан следующей схемой в расширенном алфавите $B = \{1, *, \alpha\} \supset A_2$:

$$\begin{cases} * \to * & (a) \\ \alpha 1 \to \cdot 11 & (b) \\ \Lambda \to \alpha & (c) \end{cases}$$

Пример работы этого алгоритма:

$$11111 \xrightarrow{c} \alpha 11111 \xrightarrow{b} 111111.$$

Первая подстановка схемы $* \to *$, как и в предыдущем примере, обеспечивает неприменимость данного алгоритма к словам, содержащим букву $*$, т.е. к словам, отличным от чисел $\overline{n}$.

ПРИМЕР 4.1.14. Построить нормальный алгоритм A_m^n над алфавитом $A_2 = \{1, *\}$, вычисляющий функцию-проектор $I_m^n(x_1, x_2, \ldots, x_n) = x_m$ (где $1 \le m \le n$) для любых $x_1, x_2, \ldots, x_n \in N$. Он также должен быть применим к тем и только тем словам в алфавите A_2, которые имеют вид $\overline{(k_1, k_2, \ldots, k_n)}$, причём, $A_m^n(\overline{(k_1, k_2, \ldots, k_n)}) = \overline{k_m}$ для любой упорядоченной n-ки натуральных чисел $(k_1, k_2, \ldots, k_n)$.

Требуемый алгоритм будем строить в следующем расширенном алфавите $B = \{1, *, \alpha_1, \alpha_2, \ldots, \alpha_{2n}\} \supset A_2$. Обозначим через F_i (при $1 \le i < n$) следующий список формул подстановки:

$$\begin{gathered} \alpha_1 * \to \cdot * \alpha_2 \\ \alpha_2 * \to \cdot * \alpha_3 \\ \ldots \\ \alpha_n * \to \cdot * \alpha_{n+1} \end{gathered}$$

Для удаления оставшихся частей слова используются дополнительные символы.

1.4 Рекурсивные функции

Происхождение рекурсивных функций. Всякий алгоритм однозначно сопоставляет допустимым начальным данным результат. Это означает, что с каждым алгоритмом однозначно связана функция, которую он вычисляет. В предыдущей главе II были рассмотрены примеры функций, которые вычисляются с помощью алгоритма, названного машиной Тьюринга. Возникает естественный вопрос, для всякой ли функции существует вычисляющий её алгоритм. Учитывая сформулированный в §2.4 тезис Тьюринга, утверждающий, что функция вычислима с помощью какого-нибудь алгоритма тогда и только тогда, когда она вычислима с помощью машины Тьюринга, данный вопрос трансформируется в следующий. Для всякой ли функции можно указать вычисляющую её машину Тьюринга? Если нет, то для каких функций существует вычисляющий их алгоритм (машина Тьюринга), как описать такие, как говорят, алгоритмически или эффективно вычислимые функции?

Исследование этих вопросов привело к созданию в 30-х годах XX века теории рекурсивных функций. При этом класс вычислимых функций (названных здесь рекурсивными) получил такое описание, которое весьма напоминало процесс построения аксиоматической теории на базе некоторой системы аксиом. Сначала были выбраны простейшие функции, эффективная вычислимость которых была очевидна (своего рода "аксиомы"). Затем сформулированы некоторые правила (названные операторами), на основе которых можно строить новые функции из уже имеющихся (своего рода "правила вывода"). Тогда требуемым классом функций будет совокупность всех функций, получающихся из простейших с помощью выбранных операторов. Наша цель будет состоять в том, чтобы доказать, что этот класс функций совпадает с классом функций, вычислимых с помощью машин Тьюринга.

Идея доказательства этого утверждения в одну сторону проста: сначала доказать вычислимость по Тьюрингу простейших функций, а затем — вычислимость по Тьюрингу функций, получающихся из вычислимых по Тьюрингу функций с помощью выбранных операторов.

Простейшие функции. Приступим к построению класса рекурсивных функций в соответствии с изложенным принципом. Напомним, что рассматриваются функции, заданные на множестве натуральных чисел и принимающие натуральные значения. Функции предполагаются частичными, то есть определенные, вообще говоря, не для всех значений аргументов. В качестве исходных, простейших, функций выберем следующие:

$$S(x) = x + 1 \quad \text{(функция следования)},$$
$$O(x) = 0 \quad \text{(нуль-функция)},$$
$$I_m^n(x_1, x_2, \ldots, x_n) = x_m \quad \text{(функция-проектор, } 1 \leq m \leq n\text{)}.$$

Вычислимость (более того, равномерная вычислимость) этих функций с помощью машин Тьюринга установлена в примерах 2.2.1, 2.4.4 и 2.4.5 предыдущей главы.

Далее, в качестве операторов, с помощью которых будет строиться новые функции, выберем следующие три: оператор суперпозиции, оператор примитивной рекурсии и оператор минимизации.

Оператор суперпозиции. Понятие суперпозиции функций и сложной функции хорошо известно, в частности, из курса математического анализа. Мы напомним это понятие в §2.4: там мы дали определение суперпозиции функций (определение 2.4.6) и доказали, что функция, получающаяся с помощью оператора су-

перпозиции из функций, (равномерно) вычислимых по Тьюрингу, сама (равномерно) вычислима по Тьюрингу (теорема 2.4.7). Если функция $\varphi(x_1, \ldots, x_n)$ получена из функций $f, g_1, g_2, \ldots, g_m$ с помощью оператора суперпозиции, то будем писать: $\varphi = S(f; g_1, g_2, \ldots, g_m)$, где S — оператор суперпозиции.

Свойства оператора суперпозиции:

1. Сохранение всюду определённости функций, т.е. если $f, g_1, g_2, \ldots, g_m$ — всюду определены, то и $\varphi = S(f; g_1, g_2, \ldots, g_m)$ — всюду определены. [Если хотя бы одно из значений $f, g_1, g_2, \ldots, g_m$ не определено, то и значение φ не определено].

2. Сохранение алгоритмической вычислимости функций, т.е. если $f, g_1, g_2, \ldots, g_m$ — вычислимы, то и $\varphi = S(f; g_1, g_2, \ldots, g_m)$ — вычислима.

Оператор примитивной рекурсии. ОПРЕДЕЛЕНИЕ 3.1.1. Говорят, что $(n+1)$-местная функция φ получена из n-местной функции f и $(n+2)$-местной функции g с помощью оператора примитивной рекурсии, если для любых $x_1, \ldots, x_n, y$ справедливы равенства:

$$\varphi(x_1, \ldots, x_n, 0) = f(x_1, \ldots, x_n),$$
$$\varphi(x_1, \ldots, x_n, y+1) = g(x_1, \ldots, x_n, y, \varphi(x_1, \ldots, x_n, y)).$$

Пара этих равенств называется схемой примитивной рекурсии. Обозначение: $\varphi = R(f, g)$, где R — оператор примитивной рекурсии.

Здесь важно отметить то, что независимо от числа аргументов в φ рекурсия ведётся только по одной переменной y; остальные n переменных $x_1, \ldots, x_n$ на момент применения схемы примитивной рекурсии зафиксированы и играют роль параметров. Кроме того, схема примитивной рекурсии выражает каждое значение определяемой функции φ не только через данные функции f и g, но и через так называемые предыдущие значения определяемой функции φ: прежде чем получить значение $\varphi(x_1, \ldots, x_n, k)$, придётся проделать $k+1$ вычисление по указанной схеме — для $y = 0, 1, \ldots, k$.

СВОЙСТВА ОПЕРАТОРА ПРИМИТИВНОЙ РЕКУРСИИ:

1. Сохранение всюду определённости функций, т.е. если f, g — всюду определены, то и $\varphi = R(f, g)$ — всюду определена.

2. Сохранение алгоритмической вычислимости, т.е. если f, g — вычислимы, то и $\varphi = R(f, g)$ — вычислима.

Доказательство. Пусть функции f, g вычислимы. Тогда:

$$\varphi(a, 0) = f(a) = b_0,$$
$$\varphi(a, 1) = g(a, 0, \varphi(a, 0)) = g(a, 0, b_0) = b_1,$$
$$\varphi(a, 2) = g(a, 1, \varphi(a, 1)) = g(a, 1, b_1) = b_2,$$

Таким образом, φ — вычислима. □

Понятие примитивно рекурсивной функции. Класс примитивно рекурсивных функций строится генетическим образом. Сначала выбираются первоначальные (или простейшие) функции. В качестве них берутся: $S(x) = x + 1$ — функция следования; $O(x) = 0$ — нуль-функция; $I_m^n(x_1, \ldots, x_n) = x_m$ — функции-проекторы.

ОПРЕДЕЛЕНИЕ 3.1.6. Функция называется **примитивно рекурсивной** (ПРФ), если она может быть получена из простейших функций S, O, I_m^n с помощью конечного числа применений операторов суперпозиции и примитивной рекурсии.

Таким образом, класс ПРФ представляет собой замыкание совокупности первоначальных функций S, O, I_m^n относительно операторов суперпозиции и примитивной рекурсии. Отсюда вытекает простое, но важное свойство класса ПРФ.

СВОЙСТВО КЛАССА ПРФ: класс ПРФ замкнут относительно операторов суперпозиции и примитивной рекурсии. Другими словами, функция, полученная в результате суперпозиции, а также по схеме примитивной рекурсии из примитивно рекурсивных функций сама является примитивно рекурсивной.

Отметим в заключение, что схема примитивной рекурсии задания функций устроена так, что она позволяет последовательно вычислять значение определяемой функции при каждом натуральном n. Это означает, в частности, что любая примитивно рекурсивная функция непременно всюду определена.

Наконец, введём заключительный, третий, оператор.

Оператор минимизации. ОПРЕДЕЛЕНИЕ 3.1.7. Будем говорить, что n-местная функция φ получается из $(n+1)$-местных функций f_1 и f_2 с помощью **оператора минимизации**, или **оператора наименьшего числа**, если для любых $x_1, \ldots, x_n$ значение $\varphi(x_1, \ldots, x_n)$ равно наименьшему значению аргумента y, при котором выполняется равенство $f_1(x_1, \ldots, x_n, y) = f_2(x_1, \ldots, x_n, y)$ и все предыдущие значения $f_i(x_1, \ldots, x_n, 0), \ldots, f_i(x_1, \ldots, x_n, y-1)$ $(i = 1, 2)$ определены и попарно неравны. Используется следующее обозначение:

$$\varphi(x_1, \ldots, x_n) = \mu y[f_1(x_1, \ldots, x_n, y) = f_2(x_1, \ldots, x_n, y)].$$

В частном случае может быть $f_2 = 0$. Тогда

$$\varphi(x_1, \ldots, x_n) = \mu y[f(x_1, \ldots, x_n, y) = 0].$$

Оператор минимизации называется также μ-оператором.

Этот оператор предназначен для определения следующего подкласса рекурсивных функций.

Понятие частично рекурсивной и общерекурсивной функции. Класс частично рекурсивных функций также строится генетическим образом.

ОПРЕДЕЛЕНИЕ 3.1.8. Функция называется **частично рекурсивной** (ЧРФ), если она может быть получена из простейших функций S, O, I_m^n с помощью конечного числа применений операторов суперпозиции, примитивной рекурсии и μ-оператора. Если функция всюду определена и частично рекурсивна, то она называется **общерекурсивной** (ОРФ).

Таким образом, класс ЧРФ представляет собой замыкание совокупности первоначальных функций S, O, I_m^n относительно операторов суперпозиции, примитивной рекурсии и минимизации. Отсюда вытекает простое, но важное свойство класса ЧРФ.

1.5 Эквивалентность данных формальных моделей алгоритмов

Взято из 4.2. Рекурсивные функции и нормально вычислимые функции

Эквивалентность различных теорий алгоритмов. Итак, в двух последних главах мы познакомились с тремя теориями, каждая из которых уточняет понятие

алгоритма и доказали, что все эти теории равносильны между собой. Другими словами, они описывают один и тот же класс функций, т.е. справедлива следующая теорема.

ТЕОРЕМА 4.2.5. Следующие классы функций (заданных на множестве натуральных чисел и принимающих значения в этом же множестве) совпадают:

а) класс Выч.Т всех функций, вычислимых по Тьюрингу;

б) класс ЧРФ всех частично рекурсивных функций;

в) класс Норм.выч. всех нормально вычислимых функций.

$$\text{Выч.Т} = \text{ЧРФ} = \text{Норм.выч.} \qquad \square$$

Полезно уяснить смысл и значение этого важного результата. В разное время в разных странах учёные независимо друг от друга, изучая интуитивное понятие алгоритма и алгоритмической вычислимости, создали теории, описывающие данное понятие, которые оказались равносильными. Этот факт служит мощным косвенным подтверждением адекватности этих теорий опыту вычислений, справедливости каждого из тезисов Тьюринга, Чёрча и Маркова. В самом деле, ведь если бы один из этих классов оказался шире какого-либо другого класса, то соответствующий тезис Тьюринга, Чёрча или Маркова был бы опровергнут. Например, если бы класс нормально вычислимых функций оказался шире класса рекурсивных функций, то существовала бы нормально вычислимая, но не рекурсивная функция. В силу её нормальной вычислимости она была бы алгоритмически вычислима в интуитивном понимании алгоритма, и предложение о её нерекурсивности опровергало бы тезис Чёрча. Но классы Тьюринга, Чёрча и Маркова совпадают, и таких функций не существует. Это и служит ещё одним косвенным подтверждением тезисов Тьюринга, Маркова и Чёрча.

Можно отметить, что существуют ещё и другие теории алгоритмов (одним из важных является так называемое λ-исчисление), и для всех них также доказана их равносильность с рассмотренными теориями.

1.6 Понятие об алгоритмической неразрешимости. Примеры алгоритмически неразрешимых проблем

Взято из Глава VII АЛГОРИТМИЧЕСКИ НЕРАЗРЕШИМЫЕ МАССОВЫЕ ПРОБЛЕМЫ

Алгоритмически неразрешимые массовые проблемы, связанные с машинами Тьюринга.

Напомним, что в параграфе 5.1 мы осуществили эффективную нумерацию всех мыслимых машин Тьюринга, т.е. такую нумерацию, что по номеру машины мы можем восстановить саму машину. Эта нумерация будет существенно использована при построении примера функции, невычислимой по Тьюрингу.

Существование невычислимых по Тьюрингу функций. Сначала докажем чистую теорему существования.

ТЕОРЕМА 7.1.1. Существуют функции, не вычислимые по Тьюрингу, т.е. невычислимые ни на одной машине Тьюринга.

Доказательство. Функции, о которых идет речь, представляют собой функции, заданные и принимающие значения в множестве слов в алфавите $A_1 = \{1\}$. Известно, что множество слов в алфавите $A_1 = \{1\}$ счётно, т.е. рассматривается множество

всех функций, заданных на счётном множестве и принимающих значения в счётном же множестве. Как известно, это множество имеет мощность континуума. С другой стороны, поскольку множество всевозможных машин Тьюринга, как мы установили в параграфе 5.1, перечислимо, их счётно, поэтому и множество функций, вычислимых по Тьюрингу, также счётно. Континуальная мощность строго больше счётной. Следовательно, существуют функции, не вычислимые по Тьюрингу. □

Принимая во внимание тезис Тьюринга, получаем:

СЛЕДСТВИЕ 7.1.2. (С учётом тезиса Тьюринга.) Существуют невычислимые функции.

Доказанные теорема и следствие из неё являются так называемыми чистыми теоремами существования. Интересно получить пример конкретной функции, не вычислимой по Тьюрингу (и, следовательно, не вычислимой).

Пример не вычислимой по Тьюрингу функции. Укажем конкретную функцию, которую нельзя вычислить ни на какой машине Тьюринга. На основании тезиса Тьюринга, это будет означать, что не существует вообще никакого алгоритма для вычисления значений такой функции.

Рассмотрим следующую функцию $\psi(\alpha)$, заданную на множестве слов в алфавите $A_1 = \{1\}$, определив её следующим образом. Для произвольного слова α длины n в алфавите $A_1 = \{1\}$ положим:

$$\psi(\alpha) = \begin{cases} \beta_n 1, & \text{если слово } \alpha \text{ перерабатывается машиной Тьюринга с номером } n \text{ в слово } \beta_n \text{ алфав} \\ 1, & \text{в противном случае.} \end{cases}$$

Докажем, что функция $\psi(\alpha)$ не вычислима по Тьюрингу.

ТЕОРЕМА 7.1.3. Функция $\psi(\alpha)$ не вычислима по Тьюрингу.

Доказательство. Допустим противное. Это означает, что существует машина Тьюринга T со стандартным алфавитом $\{a_0, 1, q, \text{П}, \text{Л}\}$, вычисляющая эту функцию. Пусть k — номер этой машины в нумерации, описанной в предыдущем пункте. Посмотрим, чему равно слово $\psi(1^k)$, (напомним, что $1^k = 11\ldots1$ — слово из k единиц), являющееся значением функции $\psi(\alpha)$ при $\alpha = 1^k$. Предположим, что машина T перерабатывает слово 1^k в слово β_k в том же алфавите $A_1 = \{1\}$. Тогда по определению вычислимости функции $\psi(\alpha)$ на машине T это означает, что $\psi(1^k) = \beta_k$. Но с другой стороны, по самому определению функции $\psi(\alpha)$ это означает, что $\psi(1^k) = \beta_k 1$. Полученное противоречие доказывает, что машины Тьюринга, вычисляющей функцию $\psi(\alpha)$, не существует. □

Принимая во внимание тезис Тьюринга, получаем:

СЛЕДСТВИЕ 7.1.4. (С учётом тезиса Тьюринга.) Функция $\psi(\alpha)$ алгоритмически не вычислима, т.е. не существует вообще никакого алгоритма для вычисления значений функции $\psi(\alpha)$.

Это означает, что массовая проблема нахождения значений функции $\psi(\alpha)$ для всевозможных значений аргумента алгоритмически неразрешима.

Проблема распознавания самоприменимости. Рассмотрим ещё два примера неразрешимых алгоритмических проблем — распознавания самоприменимости и распознавания применимости машин Тьюринга. Сначала о первой. Предположим, что на ленте машины Тьюринга записана её собственная функциональная схема в алфавите машины. Если машина применима к такой конфигурации, то будем называть её **самоприменяемой**, в противном случае — **несамоприменяемой**. Возникает массовая проблема распознавания самоприменяемых машин Тьюринга, состоящая

в следующем. По заданной функциональной схеме (программе) машины Тьюринга установить, к какому классу относится машина: к классу самоприменяемых машин или к классу несамоприменяемых машин.

ТЕОРЕМА 7.1.5. Проблема распознавания самоприменимых машин Тьюринга алгоритмически неразрешима.

Доказательство. Допустим противное, т.е. алгоритм для такого распознавания существует. Значит, на основании тезиса Тьюринга, существует машина Тьюринга, реализующая данный алгоритм. Пусть Θ — такая машина. На её ленту заносится соответствующим образом закодированная программа той или иной машины Тьюринга. При этом, если машина самоприменима, то занесённое слово перерабатывается машиной Θ в какой-то символ σ (имеющий смысл утвердительного ответа на поставленный вопрос о самоприменимости). Если же машина не самоприменима, то занесённое на ленту слово, кодирующее её программу, перерабатывается машиной Θ в какой-то символ τ (имеющий смысл отрицательного ответа на поставленный вопрос).

Рассмотрим теперь такую машину Тьюринга Θ_1, которая по-прежнему перерабатывает несамоприменимые коды в τ, а к самоприменимым кодам машина Θ_1 уже не применима. Такая машина получается из машины Θ, если следующим образом слегка изменить её программу: после появления символа σ вместо остановки машина должна неограниченно его повторять.

Итак, Θ_1 применима ко всякому несамоприменимому коду (вырабатывая символ τ) и не применима к самоприменимым кодам. Это приводит к противоречию, потому что такая машина не может быть ни самоприменимой, ни несамоприменимой. В самом деле, если машина Θ_1 самоприменима, то она неприменима к своему коду. Значит, машина несамоприменима. Противоречие. С другой стороны, если машина Θ_1 несамоприменима, то её код должен перерабатываться самой машиной Θ_1 в символ τ. Значит, Θ_1 применима к собственному коду, т.е. самоприменима. Снова противоречие. Оно и доказывает теорему. □

Проблема распознавания применимости. На основании доказанной теоремы устанавливается алгоритмическая неразрешимость и некоторых других массовых проблем, возникающих в теории машин Тьюринга, в частности, **проблемы распознавания применимости для машин Тьюринга**, которая состоит в следующем. Заданы функциональная схема (программа) какой-нибудь машины Тьюринга и конфигурация в ней: узнать, применима ли машина к данной конфигурации или нет. Эта проблема носит также название **проблемы остановки**.

ТЕОРЕМА 7.1.6. Проблема распознавания применимых машин Тьюринга алгоритмически неразрешима.

Доказательство. Если бы существовал алгоритм для решения этой проблемы, то с его помощью можно было бы узнать, применима ли машина к слову, кодирующему её собственную программу, т.е. самоприменима ли она. Но на основании предыдущей теоремы, известно, что такого алгоритма не существует. □

7.2. Алгоритмически неразрешимые массовые проблемы в общей теории алгоритмов

В главах V и VI мы уже достаточно далеко продвинулись в развитии общей теории вычислимости, так что здесь сможем применить разработанные там методы к установлению алгоритмической неразрешимости ряда массовых проблем в общей тео-

рии алгоритмов. Первый шаг в этом направлении нами уже сделан: теорема 5.1.2 утверждает существование всюду определённой невычислимой функции. Отметим, что многие доказательства неразрешимости основываются на рассмотренной в параграфе 5.1 нумерации алгоритмов и на диагональном методе, который мы также обсуждали в параграфе 5.1.

Проблема распознавания нулевых функций. Её неразрешимость также доказывается путём сведения к ней неразрешимой проблемы распознавания самоприменимости "$x \in W_x$".

ТЕОРЕМА 7.2.4. Проблема "$\varphi_x = 0$"алгоритмически неразрешима.

Доказательство. Рассмотрим функцию f, определяемую формулой:

$$f(x,y) = \begin{cases} 0, & \text{если } x \in W_x, \\ \text{не определено, если } x \notin W_x. \end{cases}$$

Эта функция вычислима. Тогда по теореме 5.2.2 (Клини о параметризации) существует всюду определённая вычислимая функция $k(x)$ такая, что $f(x,y) = \varphi_{k(x)}(y)$. Учитывая определение функции f, отсюда получаем:

$$x \in W_x \iff \varphi_{k(x)} = 0. \qquad (*)$$

Обратимся теперь к нашей массовой проблеме "$\varphi_x = 0$"и рассмотрим её характеристическую функцию:

$$g(x) = \begin{cases} 1, & \text{если } \varphi_x = 0, \\ 0, & \text{если } \varphi_x \neq 0. \end{cases}$$

Допустим, что функция g вычислима. Тогда вычислимой будет и функция $h(x) = g(k(x))$ как суперпозиция вычислимых функций. Но эта функция с учётом равносильности (*) может быть представлена в следующем виде

$$h(x) = \begin{cases} 1, & \text{если } \varphi_{k(x)} = 0, \text{ т.е. } x \in W_x, \\ 0, & \text{если } \varphi_{k(x)} \neq 0, \text{ т.е. } x \notin W_x. \end{cases}$$

Отсюда видно, что $h(x)$ есть характеристическая функция проблемы самоприменимости $x \in W_x$, которая согласно теореме 7.2.1, алгоритмически неразрешима, а её характеристическая функция $h(x)$ невычислима.

Полученное противоречие доказывает, что функция g невычислима и, значит, характеризуемая ею массовая проблема "$\varphi_x = 0$"алгоритмически неразрешима.

Проблема распознавания нетривиальных свойств вычислимых функций. Рассмотренные ранее неразрешимые проблемы носили довольно экзотический характер: все они были так или иначе связаны с самоприменимостью алгоритма, когда алгоритм работает с собственным описанием (находится значение вычислимой функции φ_x в точке, являющейся её собственным номером в выбранной нумерации вычислимых функций: $\varphi_x(x)$). Сейчас будет доказана теорема высокой степени общности, которая раскроет перед нами целый спектр алгоритмически неразрешимых массовых проблем. Из неё последуют некоторые результаты, доказанные ранее, а при доказательстве мы снова воспользуемся $s-m-n$-теоремой Клини (о параметризации) и методом сведения рассматриваемой проблемы к проблеме самоприменимости "$x \in W_x$".

ТЕОРЕМА 7.2.12. (Райс). Пусть C — любой непустой собственный класс вычислимых функций от одного аргумента (существуют как функции, принадлежащие

C, так и вычислимые функции, не принадлежащие C). Тогда не существует алгоритма, который бы по номеру x функции φ_x определял бы, принадлежит φ_x классу C или нет. Иначе говоря, проблема "$\varphi_x \in C$"алгоритмически неразрешима. Другими словами, множество $\{x : \varphi_x \in C\}$ неразрешимо.

Доказательство. Характеристическая функция этой массовой проблемы есть характеристическая функция множества $M = \{x : \varphi_x \in C\}$:

$$\chi_M(x) = \begin{cases} 1, & \text{если } x \in M, \text{ т.е. } \varphi_x \in C, \\ 0, & \text{если } x \notin M, \text{ т.е. } \varphi_x \notin C. \end{cases}$$

Пусть g_0 — нигде не определённая функция. Рассмотрим сначала случай, когда $g_0 \notin C$. Выберем тогда какую-нибудь конкретную вычислимую функцию $g \in C$ и рассмотрим функцию $f(x, y)$, задаваемую следующим образом:

$$f(x,y) = \begin{cases} g(y), & \text{если } \varphi_x(x) \text{ опр., т.е. } x \in W_x, \\ g_0(y) \text{ (т.е. не опр.), если } \varphi_x(x) \text{ не опр., т.е. } x \notin W_x. \end{cases}$$

Функция $f(x, y)$ вычислима. Для её вычисления надо вычислять $\varphi_x(x)$: если $\varphi_x(x)$ определено, то этот процесс когда-нибудь остановится и нужно будет перейти к вычислению $g(y)$; если же $\varphi_x(x)$ не определено, то процесс не остановится, что равносильно вычислению нигде не определённой функции $g_0(y)$.

Из вычислимости функции $f(x, y)$ по теореме о параметризации ($s-m-n$-теорема Клини 5.2.1) заключаем, что существует такая всюду определённая вычислимая функция $k(x)$, что $f(x, y) = \varphi_{k(x)}(y)$. Отсюда с учётом определения функции $f(x, y)$ приходим к следующим выводам:

$$x \in W_x \implies \varphi_{k(x)} = g \implies \varphi_{k(x)} \in C;$$

$$x \notin W_x \implies \varphi_{k(x)} = g_0 \implies \varphi_{k(x)} \notin C,$$

или

$$x \in W_x \iff \varphi_{k(x)} \in C. \qquad (*)$$

Это означает, что с помощью вычислимой функции $k(x)$ мы свели проблему самоприменимости "$x \in W_x$"к исследуемой проблеме "$\varphi_x \in C$".

Предположим теперь на минуту, что наша проблема "$\varphi_x \in C$"разрешима, т.е. её характеристическая функция χ_M вычислима. Тогда вычислимой будет и функция $h(x) = \chi_M(k(x))$ (как суперпозиция вычислимых функций). Эта функция имеет вид:

$$h(x) = \begin{cases} 1, & \text{если } k(x) \in M, \text{ т.е. } \varphi_{k(x)} \in C, \\ 0, & \text{если } k(x) \notin M, \text{ т.е. } \varphi_{k(x)} \notin C. \end{cases}$$

Учитывая равносильность (*), функция h приводится к виду:

$$h(x) = \begin{cases} 1, & \text{если } x \in W_x, \\ 0, & \text{если } x \notin W_x, \end{cases}$$

т.е. это есть характеристическая функция проблемы самоприменимости "$x \in W_x$ неразрешимость которой, а также невычислимость функции h доказаны в теореме 7.2.1.

Полученное противоречие доказывает теорему. □

Отметим, что из теоремы Райса немедленно вытекает неразрешимость проблемы "$\varphi_x = 0$"(теорема 7.2.4), если в качестве C взять множество, состоящее из единственной нуль-функции 0.

Дальнейшие примеры алгоритмически неразрешимых массовых проблем. Перечислим ряд массовых проблем, алгоритмическая неразрешимость которых может быть доказана с использованием $s-m-n$-теоремы Клини о параметризации и метода диагонализации. Доказательства проводятся методом сведения каждой из этих проблем к проблеме самоприменимости "$x \in W_x$"(теорема 7.2.1), т.е. доказывается, что если бы мы смогли эффективно решить исследуемую проблему, то мы могли бы воспользоваться этим для получения эффективного метода решения проблемы самоприменимости алгоритмов.

Вот некоторые из этих проблем:

1. Проблема "$\varphi_x = c$"определения по любому x, является ли функция φ_x постоянной величиной.

2. Проблема "$x \in E_x$"определения по любому x, входит ли x в область значений функции φ_x.

3. Проблема "E_x бесконечно"определения по любому x, бесконечно ли множество значений функции φ_x.

4. Проблема "$W_x = W_y$"определения по любым x, y, совпадают ли области определения функции φ_x и φ_y.

5. Проблема "$W_x = \emptyset$"определения по любому x, является ли функция φ_x нигде не определённой.

6. Проблема "$\varphi_x(x) = 0$"определения по любому x, принимает ли функция φ_x в точке x значение 0.

7. Проблема "$\varphi_x(y) = 0$"определения по любым x, y, принимает ли функция φ_x в точке y значение 0.

8. Проблема "$\varphi_x = g$"определения по любому x, является ли функция φ_x любой фиксированной наперёд заданной вычислимой функцией g.

9. Проблема определения по любому x, является ли функция φ_x всюду определённой и постоянной.

1.7 Понятие сложности алгоритмов

Взято из 8.1. Как измерять сложность вычислительных задач и массовых проблем

Мы хотим проанализировать на сложность вычислительные задачи (вычисления), массовые проблемы и алгоритмы их решения.

С интуитивной точки зрения сложность вычисления (решения вычислительной задачи, т.е. работы данного алгоритма на заданных входных данных) определяется ресурсами, требующимися для этого вычисления. Важнейшими в этом смысле ресурсами являются время (количество шагов или тактов работы алгоритма) и объём памяти (например, наибольший номер ячейки, над которой побывала считывающая головка машины Тьюринга в процессе вычисления).

Чтобы охарактеризовать сложность конкретного алгоритма (т.е. конкретного способа решения массовой задачи), нужно принять во внимание работу этого алгоритма на всех допустимых начальных данных.

Наконец, измерить сложность массовой проблемы — задача наиболее трудная, и решается она разными способами. В конце настоящего параграфа мы обсудим один, на первый взгляд, естественный способ характеризации сложности массовой проблемы и установим его несостоятельность.

1.8 Классы P и NP

Взято из 8.3. Основы теории NP-полных массовых проблем

Детерминированные алгоритмы и класс P. Термином "детерминированные алгоритмы"мы будем называть обычные алгоритмы, с которыми мы имели дело до сих пор, с тем, чтобы подчеркнуть их разницу с алгоритмами недетерминированными, которые будут определены и рассмотрены в следующем пункте. В силу тезиса Тьюринга, любой (детерминированный) алгоритм можно проинтерпретировать в виде машины Тьюринга, которую также в этом случае будем называть **детерминированной**.

Применительно к задачам распознавания слегка модифицируем понятие детерминированной машины Тьюринга (ДМТ). Будем считать, что в алфавите Q её внутренних состояний вместо одного заключительного состояния q_0 имеется два заключительных состояния q_Y и q_N (от слов Yes — ДА, No — НЕТ).

Будем говорить, что ДМТ M, имеющая входной алфавит A, **принимает** входное слово $x \in A^*$, если, будучи применённой ко входу x, она останавливается в состоянии q_Y. Совокупность всех таких слов в алфавите A, которые предусматривают ответ ДА (остановка в состоянии q_Y), образует язык, который обозначим L_M и будем называть языком, **распознаваемым** машиной M: $L_M = \{x \in A^* : M \text{ принимает } x\}$.

Соответствие между "распознаванием"языков и "решением"проблем следующим образом. Будем говорить, что ДМТ M **решает проблему распознавания** Π со схемой кодирования e, если M останавливается на всех словах, составленных из букв входного алфавита A, и $L_M = L[\Pi, e]$.

ПРИМЕР 8.3.9. Рассмотрим ДМТ с внешним алфавитом $A = \{a, b\}$, алфавитом внутренних состояний $Q = \{q_1, q_2, q_3, q_4, q_Y, q_N\}$ и программой:

	a	b	Λ
q_1	$q_1 a \Pi$	$q_1 b \Pi$	$q_2 \Lambda Л$
q_2	$q_3 \Lambda Л$	$q_4 \Lambda Л$	$q_N \Lambda Л$
q_3	$q_Y \Lambda Л$	$q_N \Lambda Л$	$q_N \Lambda Л$
q_4	$q_N \Lambda Л$	$q_N \Lambda Л$	$q_N \Lambda Л$

Таблица 1: *
(Λ — символ пустой ячейки).

Посмотрим, какое слово выдаст и в каком состоянии остановится машина, начав работать со слова $q_1 babaa$:

$q_1 babaa \Longrightarrow bq_1 abaa \Longrightarrow baq_1 baa \Longrightarrow babq_1 aa \Longrightarrow babaq_1 a \Longrightarrow babaaq_1 \Lambda \Longrightarrow babaaq_2 a\Lambda \Longrightarrow b$

Проверьте самостоятельно, что $q_1 baba \Longrightarrow bq_N a\Lambda\Lambda\Lambda$; $q_1 bab \Longrightarrow q_N b\Lambda\Lambda\Lambda$; $q_1 babab \Longrightarrow b_1 aq_N \Lambda\Lambda\Lambda$.

Докажите, что данная машина Тьюринга распознаёт язык, состоящий из тех и только тех слов алфавита $A = \{a, b\}$, которые оканчиваются двумя буквами a. □

Здесь мы можем детализировать понятие вычислительной сложности алгоритма, представляемого детерминированной машиной Тьюринга (ДМТ) M. Мерой этой сложности выступает функция $T_M : N^+ \to N^+$ такая, что $T_M(n)$ есть максимальное число шагов, требуемое для вычисления на машине M результата для всевозможных входных слов $x \in A^*$ длины n. Если эта функция ограничена некоторым полиномом $p(n)$, т.е. $T_M(n) \leq p(n)$ для всех $n \in N^+$, то детерминированная машина Тьюринга M называется **полиномиальной**.

Говорят, что формальный язык L принадлежит классу $\mathbf{P}$, если он распознаётся некоторой полиномиальной детерминированной машиной Тьюринга, т.е.

$$\mathbf{P} = \{L : (\exists M)(M \text{ — полиномиальная ДМТ и } L_M = L)\}.$$

Говорят, что проблема распознавания П принадлежит классу $\mathbf{P}$ при схеме кодирования e, если язык этой проблемы при этом кодировании принадлежит классу $\mathbf{P}$, т.е. $L[\Pi, e] \in \mathbf{P}$; или существует полиномиальная ДМТ, которая решает проблему П при кодировании e.

Таким образом, снова принимая тезис Тьюринга, можно сказать, что класс $\mathbf{P}$ — это класс таких массовых проблем распознавания, каждая из которых может быть решена полиномиальным алгоритмом. (Название класса происходит от Polynomial — полиномиальный).

Отметим, что сложностные классы $\mathbf{P}$ и $\mathbf{NP}$ алгоритмических проблем были независимо введены в работах Кобхэма (Alan Cobham, 1964) и Эдмондса (Jack Edmonds, 1965), которые стремились к тому, чтобы дать теоретический ответ на вопрос, какие алгоритмические проблемы могут быть эффективно решены на компьютере, т.е. решены за реальное (разумное) время.

Недетерминированные алгоритмы и класс NP. В этом пункте введём в рассмотрение важный класс языков (а вместе с ними — и соответствующих проблем распознавания свойств) — класс $\mathbf{NP}$, который будет включать класс $\mathbf{P}$.

Предварительно определим понятие недетерминированной машины Тьюринга (НДМТ). В отличие от ДМТ, которая в каждый момент времени может выполнять какое-либо одно действие, НДМТ из любого данного состояния может переходить в более одного допустимого следующего состояния, и в каждый данный момент времени может выполнять более одного действия. Можно также сказать, что недетерминированный алгоритм производит вычисления до тех пор, пока не доходит до места, в котором должен быть сделан выбор из нескольких альтернатив. Детерминированный алгоритм исследовал бы в этом случае одну альтернативу, а потом бы возвращался для исследования другой альтернативы. Недетерминированный алгоритм может исследовать все возможности одновременно, как бы "копируя"самого себя для каждой альтернативы. Все копии работают независимо, не сообщаясь друг с другом каким-либо образом. Эти копии, конечно, могут создавать дальнейшие копии и т.д. Если копия обнаруживает, что она сделала неправильный (или безрезультатный) выбор, она прекращает выполняться. Если копия находит решение, она объявляет о своем успехе, и все копии прекращают работать.

Итак, недетерминированный алгоритм характеризуется двухшаговым подходом к решению задачи. На первом шаге имеется недетерминированный алгоритм, генерирующий возможное решение такой задачи — что-то вроде попытки угадать решение. Иногда такая попытка оказывается успешной, и мы получаем оптимальный или

близкий к оптимальному ответ, иногда — безуспешной: ответ далёк от оптимального. На втором шаге проверяется, действительно ли ответ, полученный на первом шаге, является решением исходной задачи. Если каждый из этих шагов по отдельности требует полиномиального времени, то алгоритм называется недетерминированным полиномиальным.

НДМТ также обладает алфавитом символов на ленте $A = \{a_1, a_2, \ldots, a_m\}$, алфавитом внутренних состояний $Q = \{q_1, q_2, \ldots, q_n, q_Y, q_N\}$. Но в отличие от ДМТ, ее команды имеют вид списка:

$$q_i a_j \to q_{k_1} a_{l_1} X_1, q_{k_2} a_{l_2} X_2, \ldots, q_{k_r} a_{l_r} X_r.$$

Будем говорить, что НДМТ M **принимает** x, если, по меньшей мере, одно из вычислений на входе x является принимающим. Совокупность всех слов x, которые данная НДМТ M принимает, образуют язык, называемый языком, **распознаваемым** машиной: $L_M = \{x \in A^* : M \text{ принимает } x\}$.

Временем, требующимся НДМТ M для того, чтобы принять слово $x \in L_M$, будем считать минимальное число шагов выполнения машины M до достижения заключительного состояния q_Y, где минимум берётся по всем принимающим вычислениям машины M со входом x.

Временная сложность НДМТ M — это функция $T_M : N^+ \to N^+$, определяемая следующим образом:

$$T_M(n) = \max(\{1\} \cup \{m : (\exists x \in L_M)(|x| = n \text{ и время принятия } x \text{ машиной } M \text{ равно } m\})).$$

Заметим, что эта функция зависит только от числа шагов, выполняемых машиной в принимающих вычислениях. Если же нет ни одного входа длины n, принимаемого машиной M, то полагаем $T(n) = 1$.

НДМТ называется **НДМТ с полиномиальным временем работы**, если найдется полином p такой, что $T_M(n) \leq p(n)$ при всех $n \geq 1$.

Теперь можем формально определить класс **NP** как класс языков, распознаваемых недетерминированными машинами Тьюринга с полиномиальным временем работы:

$$\mathbf{NP} = \{L : \text{существует НДМТ } M \text{ с полиномиальным временем работы и } L_M = L\}.$$

В силу соответствия между формальными языками и проблемами распознавания будем говорить, что проблема распознавания Π принадлежит классу **NP** при схеме кодирования e, если язык этой проблемы при этом кодировании принадлежит классу **NP**, т.е. $L[\Pi, e] \in \mathbf{NP}$.

Взаимоотношения между классами P и NP. Поскольку понятие ДМТ представляет собой частный случай понятия НДМТ с числом разветвлений $l = 1$, поэтому все проблемы класса **P** являются также и проблемами класса **NP**, т.е. $\mathbf{P} \subseteq \mathbf{NP}$.

Этот факт также вытекает и из общих соображений, поскольку всякий детерминированный алгоритм можно рассматривать как недетерминированный алгоритм, у которого отсутствует стадия угадывания, а стадия проверки представлена данным детерминированным алгоритмом.

После этого естественно возникает вопрос, совпадают ли классы **P** и **NP**, т.е. $\mathbf{P} = \mathbf{NP}$, или класс **P** является собственным подклассом в **NP**, т.е. $\mathbf{P} \neq \mathbf{NP}$. Ответ на этот вопрос к настоящему времени не известен. Другими словами, не известна

ни одна проблема из класса **NP**, которая не входила бы в класс **P**, т.е. которая решалась бы недетерминированным полиномиальным алгоритмом, но не может быть решена ни одним детерминированным полиномиальным алгоритмом. Эта проблема считается важнейшей в науке о вычислениях и выделена в ряд важнейших общематематических проблем XXI века. Её значимость обусловлена тем, что огромное количество "естественных"массовых проблем входят в класс **NP** в то время, как эффективно решаемые проблемы образуют класс **P**.

В пользу утверждения $\mathbf{P} \neq \mathbf{NP}$ говорят многие доводы. В частности, полиномиальные недетерминированные алгоритмы определённо оказываются более мощными, чем полиномиальные детерминированные алгоритмы, и не известны общие методы их превращения в детерминированные полиномиальные алгоритмы. К настоящему времени известно, что всякая проблема из класса **NP** может быть решена детерминированным алгоритмом, но с экспоненциальной сложностью $O(2^{p(n)})$, где p — некоторый полином. Это говорит о способности недетерминированного алгоритма проверять за полиномиальное время экспоненциальное число возможностей, что наталкивает на мысль о значительно большей мощности таких алгоритмов по сравнению с детерминированными полиномиальными алгоритмами. Возможно, именно поэтому для многих проблем класса **NP**, таких как КОММИВОЯЖЕР, ИЗОМОРФИЗМ ПОДГРАФУ и большого числа других задач до сих пор не найдено детерминированного полиномиального алгоритма, несмотря на упорные усилия многих известных математиков. Так что вопрос, выполняется ли равенство $\mathbf{P} = \mathbf{NP}$, до сих пор остаётся предметом исследований по всему миру.

С учётом накопленного опыта и при существующем в настоящее время уровне знаний, по-видимому, более разумно работать при ощущении, что $\mathbf{P} \neq \mathbf{NP}$. Хотя ещё раз подчеркнём, что прямого доказательства этой гипотезы пока не существует.

1.9 Полиномиальная сводимость задач

Взято из 8.3. Основы теории NP-полных массовых проблем

Алгоритмическая сводимость массовых проблем. История развития науки показывает, что новое научное знание зачастую возникает с опорой на ранее полученные знания. Так, в математике новые теоремы доказываются с использованием теорем, доказанных ранее. В теории алгоритмов этот принцип находит своё выражение в сведении вопроса об алгоритмической разрешимости или неразрешимости одной массовой проблемы к аналогичному вопросу для другой массовой проблемы. Так, неразрешимость проблем остановки алгоритмов, распознавания нулевых функций, равенства двух вычислимых функций и т.д. была доказана сведением этих проблем к проблеме самоприменимости алгоритмов, неразрешимость которой была установлена раньше. Аналогично и для разрешимых массовых проблем. Например, массовая проблема решения биквадратных уравнений $x^4 + ax^2 + b = 0$ подстановкой $t = x^2$ сводится к алгоритмически разрешимой массовой проблеме решения квадратных уравнений $t^2 + at + b = 0$ и, следовательно, сама оказывается разрешимой.

Дадим определение алгоритмической сводимости одной массовой проблемы к другой. Рассмотрим две массовые проблемы $\Pi_1 = \{Z_1^i\}$ и $\Pi_2 = \{Z_2^j\}$, состоящие из единичных задач $Z_1^i, i = 1, 2, \ldots, Z_2^j, j = 1, 2, \ldots$. Каждая из задач Z_1^i и Z_2^j имеет свои исходные данные и свой ответ, находимый в процессе решения задачи. Пусть A_1 и A_2 — алгоритмы, решающие массовые проблемы Π_1 и Π_2 соответственно.

Предположим теперь, что между массовой проблемой Π_1, т.е. между множеством

задач $\{Z_1^i\}$, и некоторым подмножеством $\hat{\Pi}_2$ множества задач массовой проблемы Π_2 установлено определённое соответствие

$$\Pi_1 = \{Z_1^i\} \quad \longrightarrow \quad \{Z_2^j\} = \hat{\Pi}_2 \subseteq \Pi_2,$$

задаваемое алгоритмами A_1' и A_2': алгоритм A_1' по исходным данным задачи $Z_1^i \in \Pi_1$ находит исходные данные соответствующей задачи $Z_2^j \in \hat{\Pi}_2$, а алгоритм A_2' по ответу задачи Z_2^j находит ответ задачи Z_1^i.

Если такие алгоритмы A_1' и A_2' существуют, то говорят, что массовая проблема Π_1 **сводится** к массовой проблеме Π_2 (обозначение: $\Pi_1 \leq \Pi_2$). При этом алгоритмы A_1' и A_2' называются **алгоритмами сведения**.

Если удалось установить сводимость $\Pi_1 \leq \Pi_2$, то это означает следующее:

- если проблема Π_2 алгоритмически разрешима, то и проблема Π_1 алгоритмически разрешима;
- если проблема Π_1 алгоритмически неразрешима, то и проблема Π_2 алгоритмически неразрешима.

В теории сложности вычислений особое значение имеет **полиномиальная сводимость**. Говорят, что массовая проблема Π_1 полиномиально сводится к массовой проблеме Π_2 ($\Pi_1 \overset{P}{\to} \Pi_2$), если:

1. существует алгоритм A_1', который по исходным данным задачи $Z_1^i \in \Pi_1$ строит исходные данные задачи $Z_2^j \in \hat{\Pi}_2$ за время, ограниченное полиномом от размера исходных данных Z_1^i;
2. существует алгоритм A_2', который по ответу задачи Z_2^j находит ответ задачи Z_1^i за время, ограниченное полиномом от размера ответа Z_2^j.

Если массовая проблема Π_1 полиномиально сводится к массовой проблеме Π_2, и проблема Π_2 имеет полиномиальную сложность, то и массовая проблема Π_1 также имеет полиномиальную сложность. Это объясняется тем, что последовательное исполнение трёх полиномиальных алгоритмов A_1', A_2 и A_2' является алгоритмом полиномиальной сложности. Соответственно, если массовая проблема Π_1 полиномиально сведена к массовой проблеме Π_2 и не существует полиномиального алгоритма решения массовой проблемы Π_1, то не существует полиномиального алгоритма решения и массовой проблемы Π_2.

Нетрудно понять, что отношения полиномиальной сводимости обладает свойством транзитивности: если проблема Π_1 полиномиально сводится к Π_2, а проблема Π_2 полиномиально сводится к Π_3, то Π_1 полиномиально сводится к Π_3, так как последовательное выполнение двух полиномиальных алгоритмов является очевидно полиномиальным алгоритмом, т.е. если $\Pi_1 \overset{P}{\to} \Pi_2$, $\Pi_2 \overset{P}{\to} \Pi_3$, то $\Pi_1 \overset{P}{\to} \Pi_3$.

Приведём простой пример сведения одной массовой проблемы к другой. Пусть первая проблема состоит в том, чтобы выяснить, принимает ли одна из пропозициональных переменных $X_1, X_2, \ldots, X_n$ значение "истина и при положительном решении выдать ответ ДА, а при отрицательном — НЕТ. Вторая задача заключается в том, чтобы найти максимальное значение в списке целых чисел. Каждая из них допускает простое и ясное решение, но предположим на минуту, что мы знаем решение задачи о поиске максимума, а задачу про пропозициональные переменные решать не

умеем. Мы хотим свести задачу о пропозициональных переменных к задаче о максимуме целых чисел. Напишем алгоритм преобразования набора значений пропозициональных переменных в список целых чисел, который значению "ложь"сопоставляет число 0, а значению "истина— число 1. Затем воспользуемся алгоритмом поиска максимального элемента в списке. По тому, как составлялся список, заключаем, что этот максимальный элемент может быть либо нулём, либо единицей. Такой ответ можно преобразовать в ответ в задаче о пропозициональных переменных, выдавая ответ ДА, если максимальное значение равно 1, и НЕТ, если оно равно 0.

Мы видели в примере 8.2.2, что поиск максимального элемента массива выполняется за линейное время (правда, там отыскивался минимальный элемент, но ясно, что этот алгоритм легко переделывается в алгоритм поиска максимального элемента массива). Кроме того, сведение первой задачи ко второй тоже требует линейного времени, поэтому задачу о пропозициональных переменных тоже можно решить за линейное время.

1.10 Теорема Кука об NP-полноте задачи выполнимости булевой формулы

Взято из 8.3. Основы теории NP-полных массовых проблем

NP-полнота проблемы выполнимости для формул логики высказываний. Честь быть "первой"NP-полной проблемой выпала на долю хорошо известной из математической логики проблемы выполнимости формул алгебры высказываний. Она состоит в следующем. Дана формула алгебры высказываний $F(X_1, X_2, \ldots, X_n)$. Спрашивается, выполнима ли она, т.е. существует ли такой набор $\alpha_1, \alpha_2, \ldots, \alpha_n$ значений её пропозициональных (булевых) переменных $X_1, X_2, \ldots, X_n$, который превращает эту формулу в истинное высказывание $F(\alpha_1, \alpha_2, \ldots, \alpha_n) = 1$. (Здесь $\alpha_1, \alpha_2, \ldots, \alpha_n \in \{0, 1\}$). Такой набор значений называется **выполняющим набором**. Для определённости считают, что формула $F(X_1, X_2, \ldots, X_n)$ представлена в **конъюнктивной нормальной форме**, т.е. в виде конъюнкции дизъюнктивных одночленов (дизъюнктов), т.е. формул вида $X_1 \vee \neg X_2$, $\neg X_2 \vee X_3 \vee \neg X_6$, $X_1 \vee \neg X_3 \vee X_5$ и т.п. Назовём эту проблему ВЫПОЛНИМОСТЬ (или сокращённо, ВЫП). Заметим, что если данная формула F не представлена в конъюнктивной нормальной форме, то равносильными преобразованиями, используя известные законы алгебры логики, её к такой форме легко привести. Ввиду равносильности преобразований, полученная конъюнктивная нормальная форма будет выполнима в том и только в том случае, если выполнима исходная формула F.

Следующая фундаментальная теорема была доказана американским математиком Стивеном Куком в 1971 г.

ТЕОРЕМА 8.3.18. Проблема ВЫПОЛНИМОСТЬ есть NP-полная проблема.

Идея доказательства. Нетрудно понять, что проблема ВЫП лежит в классе **NP**. Если $F(X_1, X_2, \ldots, X_n) \equiv D_1 \wedge D_2 \wedge \cdots \wedge D_m$, где $D_1, D_2, \ldots, D_m$ — дизъюнктивные одночлены от переменных $X_1, X_2, \ldots, X_n$, то недетерминированному алгоритму для решения достаточно указать набор значений переменных $\alpha_1, \alpha_2, \ldots, \alpha_n$ и осуществить проверку того, что этот набор превращает все дизъюнкты $D_1, D_2, \ldots, D_m$ в 1. Всё это легко сделать (недетерминированным образом) за полиномиальное время. Таким образом, ВЫП $\in$ **NP**.

Доказательство второй части теоремы может быть проведено на одном из двух языков — на языке массовых проблем или на языке формальных языков, представ-

ляющих массовые проблемы.

В первом случае нужно показать, что если проблема $\Pi \in \mathbf{NP}$, то Π полиномиально сводится к проблеме ВЫП. Для этого нужно, используя лишь тот факт, что $\Pi \in \mathbf{NP}$, для каждой единичной задачи $x \in \Pi$ уметь построить такую формулу $F(x)$ логики высказываний, что x будет единичной задачей массовой проблемы распознавания Π с ответом ДА тогда и только тогда, когда формула $F(x)$ выполнима. Причём, это построение должно потребовать только полиномиального — относительно размера $|x|$ единичной задачи x — количества времени.

Во втором случае нужно представить проблему ВЫП языком $L_{\text{вып}} = L[\text{ВЫП}, e]$ для некоторой разумной схемы кодирования e и затем доказать, что для всех языков $L \in \mathbf{NP}$ имеет место полиномиальная сводимость $L \implies L_{\text{вып}}$. Для этого должна быть построена функция f, отображающая множество слов X^* в алфавите X языка L в множество слов (в некотором алфавите), которые кодируют единичные задачи из проблемы ВЫП. Однако, здесь удобно и наглядно взять не множество кодирующих слов, а множество самих единичных задач из ВЫП, т.е. фактически множество формул логики высказываний. Причём эта функция f должна обладать тем свойством, что:

$$(\forall x \in X^*)(x \in L \iff f(x) \text{ — выполнима}),$$

и единичная задача $f(x)$ может быть построена за время, ограниченное полиномом от длины $|x|$ слова x.

Доказательство теоремы Кука. Приведём доказательство второй части теоремы Кука на языке массовых проблем (т.е., как описано в предыдущем пункте — в первом случае). Предварительно проанализируем ещё раз основные понятия, связанные с формулировкой этой теоремы.

В начале пункта "Недетерминированные алгоритмы и класс NP"настоящего параграфа было описано понятие недетерминированного алгоритма и решения (в частности, за полиномиальное время) таким алгоритмом проблем распознавания. Далее, класс **NP** определён как класс таких проблем распознавания, которые могут быть решены недетерминированными алгоритмами за полиномиальное время.

Формализуем теперь следующим образом понятие алгоритма, который осуществляет стадию проверки догадки, т.е., начиная работу со входом (z, s), где z — код задачи, s — код догадки, заканчивает её ответом ДА для некоторой догадки s, если данная задача $z \in \Pi$ действительно имеет положительное решение, и работает бесконечно для любой догадки s, если данная задача $z \in \Pi$ не имеет положительного решения. В качестве такой формализации выберем следующую модификацию понятия машины Тьюринга.

Этот алгоритм можно представлять как устройство, которое преобразует символы из алфавита Σ, записанные по одному в ячейки ленты (строки). Чтение и запись происходят по одному символу за единицу времени с помощью читающе-пишущей головки, работа которой управляется программой, состоящей из команд. В исходном состоянии головка обозревает самый левый символ строки, и программа начинает выполнять свою первую команду.

Команды программы имеют вид:

$$l : \textbf{if } \sigma \textbf{ then } (\sigma'; o; l'),$$

где l и l' — номера команд (метки), σ и σ' — символы алфавита Σ, o — одно из чисел 1, 0 или -1. Выписанная выше команда имеет следующий смысл:

Если в данный момент времени обозревается символ σ, то стереть его и написать на его месте σ', сдвинуться на o позиций вправо и далее выполнять команду с меткой l'; в противном случае выполнять следующую команду.

Сдвиг на 0 или -1 разрядов вправо означает соответственно, что головка остаётся в той же позиции или сдвигается на 1 разряд влево.

Последняя команда программы имеет вид:

$$|A| : \textbf{accept},$$

где $|A|$ — длина программы, т.е. общее число команд в ней.

Будем говорить, что строка начальных данных (z, s) **принимается** алгоритмом A, если этот алгоритм, начиная в правильном положении обрабатывать данную строку, после не более $p(|z|)$ шагов приходит к последней команде $|A|$: **accept**, где $p(n)$ — полиномиальная граница для A. Если алгоритм делает $p(|z|)$ шагов и не достигает команды **accept** ("принять") или если головка сходит со строки, то будем говорить, что строка начальных данных (z, s) **отвергается**.

Тогда класс **NP** можно более формально определить как множество проблем распознавания П, для которых существуют алгоритм A проверки догадок и полином $p(n)$, такие, что z является индивидуальной задачей проблемы П с ответом ДА тогда и только тогда, когда существует такая догадка (строка) s, что $|s| \leq p(|z|)$ и алгоритм A принимает строку начальных данных (z, s).

Описанным формальным определением мы воспользуемся для доказательства второй части теоремы Кука, а именно утверждения о том, что если П — некоторая проблема из **NP**, то П полиномиально преобразуется в проблему ВЫПОЛНИМОСТЬ. Другими словами, для данной строки z (кодирующей задачу $z \in$ П) мы должны построить формулу $F(z)$ алгебры высказываний — используя только тот факт, что П $\in$ **NP**, — такую, что z является индивидуальной задачей проблемы П с ответом ДА тогда и только тогда, когда формула $F(z)$ выполнима. Для этого рассмотрим алгоритм A проверки догадок для проблемы П, описанный выше, время и память которого по нашему предположению о том, что П $\in$ **NP**, ограничены некоторым полиномом $p(n)$.

Формула $F(z)$ алгебры высказываний будет содержать следующие булевы (пропозициональные) переменные.

а) Булевы переменные $X_{ij\sigma}$ для всех $0 \leq i, j \leq p(|z|)$ и $\sigma \in \Sigma$. Переменная $X_{ij\sigma}$ будет соответствовать утверждению: j-я позиция строки в момент времени i содержит символ σ.

б) Булевы переменные Y_{ijl} для всех $0 \leq i \leq p(|z|)$, $0 \leq j \leq p(|z|) + 1$ и $1 \leq l \leq |A|$, где $|A|$ — число команд в алгоритме A. Переменная Y_{ijl} будет соответствовать утверждению: в момент времени i обозревается j-я позиция и выполняется l-я команда. Если $j = 0$ или $j = p(|z|) + 1$, то это означает, что головка сошла со строки и, следовательно, вычисление безуспешно.

Построим теперь из этих булевых переменных такую формулу $F(z)$ алгебры высказываний, что $F(z)$ выполнима тогда и только тогда, когда z — это индивидуальная задача проблемы П с ответом ДА. Если булевы переменные имеют указанный смысл, то $F(z)$ будет, по существу, утверждать, что алгоритм A, начиная работу со строкой $(z, \dots)$, в левой части которой стоит z, может в результате принять эту строку; по определению это будет означать, что существует подходящая догадка s (т.е. такая

догадка, для которой строка начальных данных (z, s) принимается алгоритмом A); и, следовательно, что z — индивидуальная задача проблемы П с ответом ДА.

Формула $F(z)$ является конъюнкцией четырёх частей (подформул): $F(z) = U(z) \wedge S(z) \wedge W(z) \wedge E(z)$.

1. Подформула $U(z)$ выражает тот факт, что в каждый момент времени i, $0 \leq i \leq p(|z|)$, в каждой позиции строки содержится ровно один символ алфавита Σ. $U(z)$ есть конъюнкция дизъюнктов:

$$U(z) = \bigwedge_{0 \leq i,j \leq p(|z|)} \left(\bigvee_{\sigma \in \Sigma} X_{ij\sigma} \wedge \bigwedge_{\sigma \neq \tau} (\neg X_{ij\sigma} \vee \neg X_{ij\tau}) \right).$$

2. Подформула $S(z)$ утверждает, что A корректно начинает работу; другими словами, в нулевой момент времени самые левые $|z|+1$ символов в строке образуют слово za_0, читающе-пишущая головка обозревает самый левый символ строки и программа переходит к выполнению первой команды алгоритма:

$$S(z) = \left(\bigwedge_{0 \leq j \leq |z|} X_{0jz(j)} \right) \wedge X_{0,|z|+1,a_0} \wedge Y_{011}.$$

   (Здесь $z(j)$ обозначает j-й символ строки z).

3. Подформула $W(z)$ утверждает, что A работает правильно в соответствии с командами программы; $W(z)$ является конъюнкцией формул $W_{ij\sigma l}$, по одной для каждых $0 \leq i \leq p(|z|)$, $1 \leq j \leq p(|z|)$, $\sigma \in \Sigma$ и $1 \leq l < |A|$, таких, что l-я команда алгоритма A имеет вид

$$l : \textbf{if } \sigma \textbf{ then } (\sigma'; o; l').$$

   Формула $W_{ij\sigma l}$ определяется следующим образом:

$$\begin{aligned} W_{ij\sigma l} = {} & (\neg X_{ij\sigma} \vee \neg Y_{ijl} \vee X_{i+1,j,\sigma'}) \wedge \\ & \wedge (\neg X_{ij\sigma} \vee \neg Y_{ijl} \vee Y_{i+1,j+o,l'}) \wedge \\ & \wedge \bigwedge_{\tau \neq \sigma} ((\neg X_{ij\tau} \vee \neg Y_{ijl} \vee X_{i+1,j,\tau}) \wedge (\neg X_{ij\tau} \vee \neg Y_{ijl} \vee Y_{i+1,j,l+1})). \end{aligned}$$

   Она означает, что если $X_{ij\sigma}$ и Y_{ijl} оба истинны, то в следующий момент времени переменные X и Y, утверждающие, что алгоритм A выполнил правильные действия, также должны быть истинными. Для последней команды алгоритма A и для каждых i, j и σ добавляется формула

$$W_{ij\sigma|A|} = (\neg X_{ij\sigma} \vee \neg Y_{ij|A|} \vee Y_{i+1,j,|A|}),$$

   утверждающая, что как только алгоритм достигает команды **accept**, он остаётся в этом положении. Кроме того, $W(z)$ содержит дизъюнкты

$$(\neg X_{ij\sigma} \vee \neg Y_{ijl} \vee X_{i+1,j',\sigma})$$

   для всех $0 \leq i \leq p(|z|)$, $\sigma \in \Sigma$, $1 \leq l \leq |A|$, $j \neq j'$, означающие, что если A обозревает позицию, отличную от j-й, то символ в j-й позиции остаётся неизменным.

4. Последняя часть формулы $F(z)$ утверждает просто, что A корректно заканчивает свою работу, т.е. программа при этом выполняет команду **accept**. Она состоит всего из одного дизъюнкта:

$$E(z) = \bigvee_{0 \le i \le p(|z|)} Y_{p(|z|),j,|A|}.$$

Этим завершается построение $F(z)$. Прежде всего заметим, что это построение требует только полиномиального — относительно $|z|$ — количества времени. Это следует из того факта, что общая длина формулы F (число вхождений литералов, умноженное на длину индексов этих литералов в формуле F) не превосходит $O(p^3(|z|))\cdot\log p(|z|)$. Поэтому, чтобы показать, что это построение является полиномиальным сведением (преобразованием) проблемы A в проблему ВЫПОЛНИМОСТЬ, остаётся доказать следующее утверждение.

УТВЕРЖДЕНИЕ. Формула $F(z)$ выполнима в том и только в том случае, если z является индивидуальной задачей проблемы П с ответом ДА.

Для доказательства необходимости предположим, что $F(z)$ выполнена. Тогда все формулы $U(z)$, $S(z)$, $W(z)$ и $E(z)$ выполнены при одном и том же наборе t значений пропозициональных (булевых) переменных. Так как на наборе t выполняется $U(z)$, то для всех i и j ровно одна переменная $X_{ij\sigma}$ должна быть истинной; пусть это означает, что j-я ячейка в момент времени i содержит символ σ. Кроме того, для всех i ровно одна из переменных Y_{ijl} истинна; пусть это означает, что в момент времени i обозревается j-я позиция строки и выполняется оператор l. Наконец, никакая переменная вида Y_{i0l} или $Y_{i,p(|z|)+1,l}$ не может быть истинной, что означает, что головка никогда не сходит со строки. Таким образом, набор значений истинности t описывает некоторую последовательность строк, положений головки и команд. Покажем, что эта последовательность образует правильное принимающее вычисление для алгоритма A при входе (z, s) для некоторой догадки s.

Так как $S(z)$ также должно выполняться на наборе t, то указанная последовательность начинается корректно: вначале первые $|z| + 1$ позиций содержат za_0, головка обозревает самую левую позицию, и начинается выполнение команды l. То, что на наборе t выполняется $W(z)$, гарантирует, что каждая следующая конфигурация получается из предыдущей в соответствии с командами программы A. Наконец, из того, что на наборе t выполняется $E(z)$, следует, что в некоторый момент времени $i \le p(|z|)$ алгоритм достигает команды **accept**. Следовательно, последовательность конфигураций является принимающим вычислением для алгоритма A при входе (z, s), где s — некоторая строка длины $\le p(|z|)$. Значит, z является индивидуальной задачей проблемы П с ответом ДА.

Обратно, если z — индивидуальная задача проблемы П с ответом ДА, то существует догадка s длины $\le p(|z|)$, для которой алгоритм A принимает (z, s). Рассмотрим соответствующее вычисление алгоритма A и определим набор значений истинности t переменных $X_{ij\sigma}$ и Y_{ijl} в соответствии с их смыслом. Легко видеть, что на этом наборе t истинна формула $F(z)$. □

1.11 Примеры NP-полных задач, подходы к их решению

3-ВЫПОЛНИМОСТЬ (3-ВЫП)

Условие. Дана конъюнктивная нормальная форма $F(X_1, X_2, \ldots, X_n) = D_1 \wedge D_2 \wedge \cdots \wedge D_m$ от переменных $X_1, X_2, \ldots, X_n$, каждый дизъюнкт которой $D_1, D_2, \ldots, D_m$

содержит точно три из этих переменных.

Вопрос. Выполнима ли формула $F(X_1, X_2, \ldots, X_n)$, т.е. существует ли двоичный набор $\alpha_1, \alpha_2, \ldots, \alpha_n$ значений переменных, превращающий F в 1: $F(\alpha_1, \alpha_2, \ldots, \alpha_n) = 1$?

ТРЕХМЕРНОЕ СОЧЕТАНИЕ (3-С)

Условие. Дано множество $M \subseteq X \times Y \times Z$, где X, Y, Z — непересекающиеся множества, содержащие одинаковое число q элементов.

Вопрос. Верно ли, что M содержит трёхмерное сочетание, т.е. подмножество M' такое, что $|M'| = q$ и никакие два различных элемента из M' не имеют ни одной равной координаты?

ВЕРШИННОЕ ПОКРЫТИЕ (ВП)

Условие. Дан граф $G = \langle V, E \rangle$ и положительное целое число k, $k \leq |V|$.

Вопрос. Имеются ли в графе G вершинное покрытие не более чем из k элементов, т.е. такое подмножество $V' \subseteq V$, что $|V'| \leq k$ и для каждого ребра $\{u, v\} \in E$ хотя бы одна из вершин u или v принадлежит V'?

КЛИКА

Условие. Дан граф $G = \langle V, E \rangle$ и положительное целое число k, $k \leq |V|$.

Вопрос. Верно ли, что G содержит некоторую клику мощности не менее k, т.е. такое подмножество $V' \subseteq V$, что $|V'| \geq k$ и любые две вершины из V' соединены ребром из E?

ГАМИЛЬТОНОВ ЦИКЛ (ГЦ)

Условие. Дан граф $G = \langle V, E \rangle$.

Вопрос. Верно ли, что G содержит гамильтонов цикл, т.е. такую последовательность $\langle v_1, v_2, \ldots, v_n \rangle$ вершин графа G, что $n = |V|$, $\{v_n, v_1\} \in E$ и $\{v_i, v_{i+1}\} \in E$ для всех i таких, что $1 \leq i \leq n - 1$?

РАЗБИЕНИЕ

Условие. Заданы конечное множество A и вес $s(a) \in \mathbb{N}$ для каждого $a \in A$.

Вопрос. Существует ли подмножество $A' \subseteq A$ такое, что $\sum_{a \in A'} s(a) = \sum_{a \in A \setminus A'} s(a)$?

NP-полнота этих проблем впервые была доказана американским математиком Р.М. Карпом (Karp) в 1972 г. В книге [49] приводятся подробные доказательства NP-полноты этих проблем методом сведения к ним проблемы ВЫПОЛНИМОСТЬ.

Таким образом, перечисленные шесть проблем, а также и все другие NP-полные проблемы столь же "трудны как и проблема о выполнимости формул логики высказываний. В настоящее время класс NP-полных проблем насчитывает сотни представителей. Среди них проблемы коммивояжера, целочисленного линейного программирования, о пересечении трёх матроидов и т.д. Они образуют класс эквивалентности, состоящий из "самых трудных"проблем из класса **NP**. По мере того как все больше и больше проблем, представляющих интерес с разных точек зрения, попадают в этот класс эквивалентности, его значимость постоянно возрастает.

1.12 Точные и приближенные комбинаторные алгоритмы

Точные алгоритмы: организация перебора

[Взято из: Глава 1. Организация полного перебора, § 1.1–1.4, с. 4–21]

Алгоритм полного перебора используют в случае, когда для решения задачи не удается разработать эффективный алгоритм решения.

Основной принцип, на котором базируются методы полного перебора вариантов, состоит в разбиении начальной задачи P_0 на подзадачи $P_1, P_2, \ldots, P_k$ с последующей попыткой разрешить каждую из этих подзадач. Данное разбиение можно представить в виде дерева, вершины которого изображают подзадачи.

При организации перебора вариантов решения основной целью является сокращение количества решаемых подзадач. Рассмотрим три стратегии отсева неперспективных вариантов ветвления:

1. по повторению (исключение повторяющихся подзадач);
2. по недопустимости (некоторая вершина является недопустимой);
3. по рекорду (если заранее можно определить, что некоторая подзадача не даст решения лучше, чем построенное алгоритмом ранее (рекорд)).

Для задачи минимизации в качестве нижней оценки (НО) и верхней оценки (ВО) подзадачи $\overline{x}$ используются соотношения:

$$HO(\overline{x}) \leq \text{опт}(\overline{x}) \leq BO(\overline{x}),$$

где $\text{опт}(\overline{x})$ — значение оптимального решения подзадачи $\overline{x}$.

Если в задаче минимизации для вершины дерева x выполняется неравенство

$$F < F(x) + HO(\overline{x}),$$

то частичное решение x не приведет к оптимальному решению. Следовательно, для вершины x можно производить отсечение по рекорду. Метод организации перебора вариантов решения с отсечениями по рекорду часто называют **методом ветвей и границ**.

Приближенные алгоритмы: основные понятия

[Взято из: Глава 2. Эвристические и приближенные алгоритмы, § 2.1, с. 68–71]

Поскольку точные полиномиальные алгоритмы решения задач, принадлежащих классу NP, неизвестны, следует рассмотреть альтернативные возможности, дающие лишь достаточно разумные ответы. Из NP-трудности задачи следует необходимость построения для ее решения эвристических или приближенных алгоритмов.

Предположим, что дана задача максимизации $F(x) \to \max$ на множестве D. Пусть $x^{\text{опт}}$ дает максимум функционалу $F(x)$. Предположим, что есть алгоритм A, который на множестве D строит другое решение $x(A) = x^A$.

Определение 2.1. Абсолютная погрешность алгоритма A на входном наборе данных определяется как величина $|F(x^{\text{опт}}) - F(x^A)|$. **Определение 2.2.** Относительная погрешность алгоритма A на входном наборе данных определяется как величина $\frac{|F(x^{\text{опт}}) - F(x^A)|}{|F(x^{\text{опт}})|}$.

Определение 2.3. Гарантированной оценкой точности решения, получаемого при помощи алгоритма A, называется:

- для задачи максимизации – нижняя оценка (inf) величины Δ_A: $P \leq \Delta_A = \frac{|F(x^A)|}{|F(x^{\text{опт}})|} \leq 1$;

- для задачи минимума – верхняя оценка (sup) величины Δ_A: $1 \leq \Delta_A = \frac{|F(x^A)|}{|F(x^{\text{опт}})|} \leq P$.

Определение 2.4. Алгоритм называется **приближенным**, если для него известна абсолютная (относительная) погрешность, не зависящая от входных данных, либо гарантированная оценка точности. Существует следующая классификация приближенных алгоритмов:

- **субоптимальный приближенный** – величина Δ_A отлична от 0 (для задач максимизации) или от ∞ (для задач минимизации);

- **ε-приближенный (PTAS)** – для любого $\varepsilon > 0$ строит решение с относительной погрешностью ε, причем трудоемкость этого алгоритма есть функция, которая зависит от длины входа и величины $1/\varepsilon$;

- **быстрый ε-приближенный (FPTAS)** – для любого $\varepsilon > 0$ строит решение с относительной погрешностью ε, причем трудоемкость этого алгоритма есть полином от длины входа и величины $1/\varepsilon$.

Жадные (градиентные) алгоритмы

[Взято из: Глава 2, § 2.2.1–2.2.4, с. 71–76]

Жадные (градиентные) алгоритмы (англ. greedy algorithm) являются наиболее популярной эвристикой для большого количества прикладных задач.

- **Задача о коммивояжере.** Задано n городов, попарно соединенных дорогами. Для каждой из них (i, j) задается стоимость проезда c_{ij}. Коммивояжер должен посетить все n городов по одному разу и вернуться в начальный город, при этом суммарная стоимость проезда должна быть минимальной. Жадный алгоритм перебирает все ребра графа в порядке возрастания их весов и добавляет ребро, если оно не образует цикл (кроме завершающего) и не является третьим, инцидентным некоторой вершине.

- **Задача о рюкзаке.** Имеется набор объектов. Для каждого объекта i заданы (a_i, c_i), где a_i – объем и c_i – стоимость. Мы хотим заполнить рюкзак объемом W так, чтобы суммарная стоимость вошедших в него предметов была максимальна. Жадный алгоритм сортирует список объектов по удельной стоимости c_i/a_i по убыванию и последовательно заполняет рюкзак.

- **Задача о суммах элементов подмножеств.** Имеется набор предметов различных размеров и некоторая верхняя положительная граница их суммарного объема L. Необходимо найти такой набор предметов, сумма размеров которых была бы наиболее близка к L. Жадный алгоритм сортирует предметы по убыванию размеров и последовательно добавляет их, если суммарный объем не превышает L.

- **Задача о раскраске графа.** Требуется определить минимальное количество цветов c, в которые можно раскрасить вершины графа так, чтобы концы каждого его ребра были окрашены в разные цвета. Жадный алгоритм последовательно рассматривает все вершины исходного графа. Первоначально номер цвета для вершины полагаем $c = 1$. Пока в графе существуют вершины, смежные с вершиной i и покрашенные цветом c, увеличим номер цвета окраски вершины i на единицу: $c = c + 1$. Окрасим вершину с номером i в цвет c.

Приближенные алгоритмы с гарантированной оценкой точности

[Взято из: Глава 2, § 2.4.1–2.4.3, с. 85–95]

Рассмотрим задачу минимизации целевого функционала $F(x)$ на множестве X. Обозначим $F^* = \min\{F(x) \mid x \in X\}$. Пусть приближенный алгоритм H находит элемент множества X со значением F_H целевого функционала. Введем в рассмотрение величину $\Delta_H = F_H/F^*$. Под гарантированной оценкой точности решения, получаемого при помощи алгоритма H, будем понимать верхнюю оценку величины Δ_H при любом наборе входных данных задачи.

- **Задача об упаковке в контейнеры.** В условиях задачи $\frac{P}{d_j} = \frac{d}{C_j} \leq d_j$ предполагается, что количество приборов m равно числу требований n, а общий директивный срок d не меньше максимальной длительности обслуживания требований $\max p_j$. Необходимо отыскать минимальное число приборов и соответствующее расписание, при котором все требования будут обслужены к директивному сроку d.

- **Задача распределения работ на конечное число одинаковых процессоров.** Рассмотрим задачу распределения работ n на одинаковых процессорах m таким образом, чтобы минимизировать загрузку максимально загруженного процессора. Пусть p_i соответствует времени выполнения i-й работы на процессоре. Оценивается алгоритм «в минимально загруженный», на каждом шаге которого работа загружается на минимально загруженный процессор. Для любой последовательности $p_1, p_2, \ldots, p_n$ данный алгоритм строит решение, в котором загрузка максимального процессора не превосходит величины $LB + \frac{m-1}{m}LB = (2 - \frac{1}{m})LB$.

- **Метрическая задача коммивояжера.** Главным условием в данном случае предполагается выполнение так называемого неравенства треугольника, когда на матрицу весов накладывается условие, согласно которому для любой тройки вершин i, j, k выполняется $C_{i,j} + C_{j,k} > C_{i,k}$. Данное соотношение выполняется в любом метрическом пространстве, например, когда вершинам соответствуют точки на плоскости.

1.13 Примеры эффективных (полиномиальных) алгоритмов

В данном разделе собраны примеры алгоритмов, время работы которых выражается полиномиальной функцией от размера входных данных. Материал опирается на классическое изложение алгоритмов поиска, сортировки и алгоритмов на графах и сетях (Cormen et al., “Алгоритмы: построение и анализ”). Для графовых алгоритмов размер входа характеризуется числами $V = |V|$ и $E = |E|$; для сортировки и поиска — числом элементов n.

Быстрые алгоритмы поиска

Линейный поиск. Задача поиска: для массива $A[1..n]$ и значения x найти индекс i, такой что $A[i] = x$, или вернуть `NIL`, если элемента нет. Линейный поиск последовательно просматривает элементы:

1. $i \leftarrow 1$;
2. пока $i \leq n$ и $A[i] \neq x$, выполнять $i \leftarrow i + 1$;
3. если $i \leq n$, вернуть i, иначе вернуть `NIL`.

Время работы в наихудшем случае: $\Theta(n)$. Линейный поиск не требует предварительной сортировки и применим к произвольным массивам.

Бинарный поиск. Если массив $A[1..n]$ отсортирован в неубывающем порядке, поиск можно ускорить до $O(\log n)$. Алгоритм:

1. $l \leftarrow 1$, $r \leftarrow n$;
2. пока $l \leq r$:
   (a) $m \leftarrow \lfloor (l + r)/2 \rfloor$;
   (b) если $A[m] = x$, вернуть m;
   (c) если $A[m] < x$, то $l \leftarrow m + 1$, иначе $r \leftarrow m - 1$;
3. вернуть `NIL`.

На каждом шаге область поиска сокращается примерно вдвое, поэтому время работы равно $O(\log n)$, а при неудачном поиске — $\Theta(\log n)$. Бинарный поиск является основой для многих эффективных алгоритмов поиска в статических отсортированных массивах.

Хеш-поиск. Для динамического множества с операциями `INSERT`, `DELETE`, `SEARCH` эффективны хеш-таблицы. В среднем при равномерном хешировании и коэффициенте заполнения $\alpha = n/m$, где m — число ячеек, математическое ожидание времени поиска равно $\Theta(1 + \alpha)$. Если $m = \Theta(n)$, то $\alpha = O(1)$, и все словарные операции в среднем выполняются за $O(1)$. В наихудшем случае поиск в хеш-таблице с цепочками может быть $\Theta(n)$, поэтому хеширование является эффективным *в среднем*, но не гарантирует полиномиально лучшую границу в худшем случае без дополнительных предположений.

Быстрые алгоритмы сортировки

Задача сортировки: по последовательности $a_1, \ldots, a_n$ построить перестановку $a'_1 \leq a'_2 \leq \cdots \leq a'_n$. Для алгоритмов, основанных только на сравнениях, существует нижняя граница $\Omega(n \log n)$; она достигается сортировкой слиянием и пирамидальной сортировкой. Алгоритмы, использующие структуру ключей, могут работать за линейное время.

Алгоритм	Наихудшее время	Комментарий
Сортировка вставками	$\Theta(n^2)$	Хороша для малых n и почти отсортированных данны
Сортировка слиянием	$\Theta(n \log n)$	Гарантированная оценка, требует $O(n)$ доп. памяти
Пирамидальная сортировка	$O(n \log n)$	Сортировка на месте, использует пирамиду
Быстрая сортировка	$\Theta(n^2)$	В среднем $O(n \log n)$; на практике часто быстрее
Сортировка подсчётом	$\Theta(n + k)$	Для целых ключей $0..k$, устойчива
Поразрядная сортировка	$\Theta(d(n + k))$	d разрядов, устойчивая промежуточная сортировка
Карманная сортировка	$O(n)$ в среднем	Для равномерно распределённых чисел на $[0, 1)$

Таблица 2: Основные алгоритмы сортировки и их сложность

Сортировка слиянием. Алгоритм использует парадигму “разделяй и властвуй”:

1. **Разделение**: массив $A[p..r]$ делится на две части $A[p..q]$ и $A[q+1..r]$, где $q = \lfloor (p+r)/2 \rfloor$;
2. **Покорение**: обе части рекурсивно сортируются;
3. **Комбинирование**: две отсортированные части сливаются процедурой `MERGE` за $\Theta(n)$.

Рекуррентное соотношение:

$$T(n) = \begin{cases} \Theta(1), & n = 1, \\ 2T(n/2) + \Theta(n), & n > 1. \end{cases}$$

Его решение: $T(n) = \Theta(n \log n)$. Сортировка слиянием асимптотически оптимальна среди сортировок сравнением.

Пирамидальная сортировка. Строится неубывающая или невозрастающая пирамида — почти полное бинарное дерево, хранимое в массиве. Для узла i:

$$\texttt{PARENT}(i) = \lfloor i/2 \rfloor, \qquad \texttt{LEFT}(i) = 2i, \qquad \texttt{RIGHT}(i) = 2i + 1.$$

Процедура `MAX_HEAPIFY` восстанавливает свойство пирамиды за $O(\log n)$, процедура `BUILD_MAX_HEAP` строит пирамиду за $O(n)$, а сама сортировка выполняет $n - 1$ извлечений максимума. Итоговое время: $O(n \log n)$. Сортировка выполняется на месте и не требует $O(n)$ дополнительной памяти, в отличие от сортировки слиянием.

Быстрая сортировка. Алгоритм также основан на декомпозиции. Процедура `PARTITION` выбирает опорный элемент x, переставляет элементы так, что слева оказываются элементы $\leq x$, а справа — $> x$, и возвращает индекс опорного элемента. Затем обе части сортируются рекурсивно. В наихудшем случае, когда разбиение каждый раз даёт части 0 и $n-1$, время равно $\Theta(n^2)$. В среднем и при рандомизированном выборе опорного элемента математическое ожидание времени равно $O(n \log n)$. На практике быстрая сортировка часто превосходит другие алгоритмы благодаря малым постоянным множителям и работе на месте.

Линейные сортировки. Сортировка подсчётом применима к целым ключам из диапазона $0..k$. Она подсчитывает частоты, затем префиксные суммы и размещает элементы в выходном массиве. Время: $\Theta(n+k)$. При $k = O(n)$ получаем $\Theta(n)$. Поразрядная сортировка выполняет d устойчивых проходов сортировки подсчётом по разрядам; время $\Theta(d(n+k))$. Карманная сортировка распределяет числа по n карманам и сортирует каждый карман; при равномерном распределении на $[0,1)$ математическое ожидание времени равно $O(n)$.

Полиномиальные алгоритмы на графах

Представление графов. Для графа $G = (V, E)$ используются два основных представления:

- **списки смежности**: для каждой вершины u хранится список $\texttt{Adj}[u]$ смежных вершин; память $\Theta(V + E)$;
- **матрица смежности**: матрица A размера $V \times V$, где $A_{ij} = 1$, если $(i, j) \in E$; память $\Theta(V^2)$.

Для разреженных графов ($E \ll V^2$) предпочтительны списки смежности; для плотных графов и быстрой проверки наличия ребра — матрица смежности.

Поиск в ширину. Поиск в ширину (`BFS`) обходит граф "волнами": сначала вершины на расстоянии 1 от источника, затем на расстоянии 2 и т.д. Используются цвета `WHITE`, `GRAY`, `BLACK`, очередь Q, массивы d и π. Основные шаги:

1. инициализировать все вершины белыми, $d[v] = \infty$, $\pi[v] = \texttt{NIL}$;
2. $d[s] = 0$, цвет s — серый, поместить s в очередь;
3. пока очередь не пуста:
   (a) извлечь u из очереди;
   (b) для каждой $v \in \texttt{Adj}[u]$, если v белая, то покрасить её в серый, $d[v] = d[u]+1$, $\pi[v] = u$, поместить v в очередь;
   (c) покрасить u в чёрный.

Время работы: $O(V + E)$. BFS находит кратчайшие пути по числу рёбер от источника s до всех достижимых вершин и строит дерево поиска в ширину. Если граф невзвешенный, BFS решает задачу кратчайшего пути из одной вершины.

Поиск в глубину. Поиск в глубину (`DFS`) идёт "вглубь" до тех пор, пока это возможно, и выполняет возврат при отсутствии неисследованных рёбер. Для каждой вершины хранятся:

- цвет $color[v]$;
- время открытия $d[v]$;
- время завершения $f[v]$;
- предшественник $\pi[v]$.

Основные шаги `DFS_VISIT(u)`:

1. $color[u] \leftarrow$ `GRAY`, $time \leftarrow time + 1$, $d[u] \leftarrow time$;
2. для каждой $v \in$ `Adj`$[u]$:
   (a) если $color[v] =$ `WHITE`, то $\pi[v] \leftarrow u$ и рекурсивно `DFS_VISIT(v)`;
3. $color[u] \leftarrow$ `BLACK`, $time \leftarrow time + 1$, $f[u] \leftarrow time$.

Время работы: $\Theta(V + E)$. DFS строит лес поиска в глубину и классифицирует рёбра на рёбра дерева, обратные, прямые и перекрёстные. DFS используется для:

- топологической сортировки ориентированного ациклического графа: вершины упорядочиваются по убыванию времени завершения $f[v]$; время $O(V + E)$;
- поиска сильно связных компонент: два прохода DFS — по графу G и по транспонированному G^T, с обработкой вершин в порядке убывания $f[v]$; время $O(V + E)$;
- проверки ацикличности: ориентированный граф ацикличен тогда и только тогда, когда DFS не находит обратных рёбер.

Минимальное остовное дерево

Пусть $G = (V, E)$ — связный неориентированный граф с весовой функцией $w : E \to \mathbb{R}$. Остовное дерево — ациклическое подмножество $T \subseteq E$, соединяющее все вершины. Минимальное остовное дерево (MST) минимизирует $w(T) = \sum_{(u,v)\in T} w(u, v)$.

Свойство разреза. Разрез $(S, V \setminus S)$ согласован с множеством рёбер A, если ни одно ребро из A не пересекает разрез. Ребро называется лёгким для разреза, если оно имеет минимальный вес среди всех рёбер, пересекающих разрез. **Теорема о безопасном ребре**: если A входит в некоторое MST, разрез согласован с A, а (u, v) — лёгкое ребро через разрез, то $A \cup \{(u, v)\}$ также входит в некоторое MST. На этом свойстве основаны жадные алгоритмы Крускала и Прима.

Алгоритм Крускала.

1. $A \leftarrow \varnothing$;
2. для каждой вершины v создать одноэлементное множество (`MAKE_SET`);
3. отсортировать рёбра по неубыванию веса;
4. для каждого ребра (u, v) в порядке возрастания веса:
   (a) если u и v принадлежат разным компонентам (`FIND_SET`), то добавить (u, v) в A и объединить компоненты (`UNION`).
5. вернуть A.

Для непересекающихся множеств используется лес с объединением по рангу и сжатием пути; время операций — $O((V+E)\alpha(V))$, где α — очень медленно растущая функция. Основное время — сортировка рёбер: $O(E \log E) = O(E \log V)$. Итог: $O(E \log V)$.

Алгоритм Прима.

1. выбрать корневую вершину r;
2. ключ $key[v] = \infty$, $\pi[v] = \texttt{NIL}$, $key[r] = 0$;
3. поместить все вершины в очередь с приоритетами Q;
4. пока Q не пуста:
   (a) извлечь вершину u с минимальным $key[u]$;
   (b) для каждой $v \in \texttt{Adj}[u]$, если $v \in Q$ и $w(u,v) < key[v]$, то $\pi[v] = u$, $key[v] = w(u,v)$.

При реализации Q бинарной пирамидой время $O(E \log V)$. При использовании фибоначчиевой пирамиды — $O(E+V \log V)$. Алгоритм Прима удобен для плотных графов и при простой реализации через матрицу смежности даёт $O(V^2)$.

Кратчайшие пути

Пусть $G = (V, E)$ — ориентированный граф с весовой функцией $w : E \to \mathbb{R}$. Вес пути — сумма весов рёбер. Кратчайший путь из s в v обозначается $\delta(s, v)$. Основные алгоритмы приведены в таблице.

Алгоритм	Допустимые веса	Время работы
BFS	все рёбра единичного веса	$O(V + E)$
Кратчайшие пути в DAG	любые веса, без циклов	$O(V + E)$
Беллман–Форд	отрицательные, без отрицательных циклов	$O(VE)$
Дейкстра	неотрицательные	$O(E \log V)$ с бинарной пирамидо
Флойд–Уоршелл	все пары, без отрицательных циклов	$O(V^3)$
Джонсон	все пары, разреженные графы	$O(V^2 \log V + VE)$

Таблица 3: Алгоритмы поиска кратчайших путей

Алгоритм Беллмана–Форда. Алгоритм работает с отрицательными весами и обнаруживает достижимые циклы отрицательного веса. Инициализация: $d[s] = 0$, $d[v] = \infty$, $\pi[v] = \texttt{NIL}$. Затем выполняется $V - 1$ проход по всем рёбрам, в каждом проходе для каждого ребра (u, v) выполняется ослабление:

$$\text{if } d[v] > d[u] + w(u,v) \text{ then } d[v] \leftarrow d[u] + w(u,v),\ \pi[v] \leftarrow u.$$

После $V - 1$ прохода, если ещё возможно ослабление, то достижим цикл отрицательного веса. Время: $O(VE)$. Если отрицательных циклов нет, по окончании $d[v] = \delta(s, v)$.

Алгоритм Дейкстры. Применим при неотрицательных весах. Поддерживается множество S вершин с окончательно известными расстояниями и очередь с приоритетами $Q = V \setminus S$. На каждом шаге из Q извлекается вершина u с минимальным $d[u]$, добавляется в S, и ослабляются все исходящие рёбра (u, v). При неотрицательных весах жадный выбор корректен. Время: $O(E \log V)$ при бинарной пирамиде и $O(E + V \log V)$ при фибоначчиевой. Алгоритм Дейкстры также можно реализовать за $O(V^2)$ без пирамиды, что удобно для плотных графов.

Кратчайшие пути в ориентированном ациклическом графе. Если граф является DAG, вершины можно топологически отсортировать и ослабить рёбра в этом порядке за один проход. Время: $O(V + E)$. Это самый быстрый вариант для DAG с произвольными весами.

Алгоритм Флойда–Уоршелла. Решает задачу для всех пар вершин. Пусть $d_{ij}^{(k)}$ — вес кратчайшего пути из i в j, промежуточные вершины которого принадлежат $\{1, \ldots, k\}$. Тогда:

$$d_{ij}^{(k)} = \min\left(d_{ij}^{(k-1)},\ d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right).$$

После $k = V$ получаем $\delta(i, j)$. Время: $\Theta(V^3)$, память: $\Theta(V^2)$. Алгоритм допускает отрицательные веса, но не отрицательные циклы. По матрице предшественников можно восстановить пути.

Алгоритм Джонсона. Для разреженных графов с отрицательными весами, но без отрицательных циклов, используется перевзвешивание рёбер. Строится новый источник s, соединяемый с каждой вершиной рёбрами веса 0; алгоритмом Беллмана–Форда вычисляются $h(v) = \delta(s, v)$. Затем веса заменяются на

$$\widehat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0.$$

После этого для каждой вершины запускается алгоритм Дейкстры. Время: $O(V^2 \log V + VE)$. Алгоритм Джонсона эффективен для разреженных графов.

Потоки в сетях, паросочетания и задача о назначениях

Максимальный поток. В транспортной сети $G = (V, E)$ с источником s, стоком t и пропускными способностями $c(u, v)$ требуется найти поток максимальной величины. Основные алгоритмы:

- метод Форда–Фалкерсона: итеративно находит увеличивающий путь в остаточной сети; время $O(E|f^*|)$, где $|f^*|$ — величина максимального потока;
- алгоритм Эдмондса–Карпа: увеличивающий путь ищется поиском в ширину; время $O(VE^2)$;
- метод проталкивания предпотока: обобщённый алгоритм за $O(V^2E)$;
- алгоритм “поднять-в-начало” (`relabel-to-front`): время $O(V^3)$.

Все эти алгоритмы полиномиальны по V и E (кроме оценки Форда–Фалкерсона через $|f^*|$, которая может быть велика при больших пропускных способностях; Эдмондс–Карп и push-relabel дают полиномиальные оценки).

Максимальное паросочетание в двудольном графе. Для двудольного графа $G = (L \cup R, E)$ максимальное паросочетание можно найти сведением к максимальному потоку: добавляются источник s, сток t, рёбра $s \to u$ для $u \in L$, рёбра $v \to t$ для $v \in R$, все пропускные способности единичные. Целочисленный максимальный поток соответствует максимальному паросочетанию. Время: $O(VE)$, если использовать Форда–Фалкерсона/Эдмондса–Карпа; алгоритм Хопкрофта–Карпа даёт $O(\sqrt{V}E)$.

Задача о назначениях. Задача о назначениях: дана матрица стоимостей $C = (c_{ij})$ размера $n \times n$; требуется найти перестановку π такую, что

$$\sum_{i=1}^{n} c_{i,\pi(i)} \to \min .$$

Это частный случай задачи о паросочетании минимальной стоимости в полном двудольном графе. Полиномиальные подходы:

- **венгерский алгоритм** (алгоритм Куна–Манкреса): время $O(n^3)$;
- сведение к **задаче о потоке минимальной стоимости**: строится двудольная сеть, ищется поток величины n минимальной стоимости; существуют полиномиальные алгоритмы;
- сведение к **взвешенному паросочетанию** в двудольном графе и использование полиномиальных алгоритмов поиска паросочетания минимального веса.

Таким образом, задача о назначениях разрешима за полиномиальное время, например за $O(n^3)$ венгерским алгоритмом.

Заключение

Рассмотренные алгоритмы показывают, что многие важные задачи поиска, сортировки, обработки графов и сетей решаются за полиномиальное время. Для сортировки сравнением нижняя граница $\Omega(n \log n)$ достигается сортировкой слиянием и пирамидальной сортировкой; использование структуры ключей позволяет получить линейные алгоритмы. Для графов базовые обходы BFS и DFS работают за $O(V + E)$ и служат основой для топологической сортировки, поиска компонент связности и сильно связных компонент. Минимальное остовное дерево находится за $O(E \log V)$ алгоритмами Крускала и Прима, кратчайшие пути — алгоритмами Беллмана–Форда, Дейкстры, Флойда–Уоршелла и Джонсона в зависимости от ограничений на веса и плотности графа. Сетевые задачи, включая максимальный поток, паросочетания и задачу о назначениях, также имеют полиномиальные алгоритмы. Эти результаты составляют практическую базу для решения прикладных задач и одновременно демонстрируют границу между полиномиально разрешимыми задачами и NP-полными задачами, рассматриваемыми в других разделах.
Материал данного раздела подготовлен по учебнику: Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 2-е изд. — М.: Издательский дом «Вильямс», 2011.

1.14 Автоматы. Эксперименты с автоматами. Алгебры регулярных выражений. Теорема Клини о регулярных языках

Взято из: Хопкрофт Дж., Мотвани Р., Ульман Дж. «Введение в теорию автоматов, языков и вычислений». — 2-е изд. — М.: Вильямс, 2002.

1. Понятие конечного автомата

[Взято из Главы 2 «Конечные автоматы», § 2.2–2.5, стр. 61–98]

В теории вычислимости конечные автоматы служат базовой математической моделью устройств с конечной памятью. Учебник вводит три основных типа конечных автоматов:

1. **Детерминированный конечный автомат (ДКА).** Формально ДКА задается пятеркой $A = (Q, \Sigma, \delta, q_0, F)$, где:

   - Q — конечное множество состояний;
   - Σ — конечное множество входных символов (алфавит);
   - $\delta : Q \times \Sigma \to Q$ — функция переходов;
   - $q_0 \in Q$ — начальное состояние;
   - $F \subseteq Q$ — множество допускающих (заключительных) состояний.

   Автомат обрабатывает цепочку символов пошагово, переходя из одного состояния в другое строго по функции δ. Если после прочтения всей цепочки автомат оказывается в допускающем состоянии, цепочка считается допущенной.

2. **Недетерминированный конечный автомат (НКА).** В отличие от ДКА, из одного состояния по одному и тому же символу НКА может переходить в несколько различных состояний. Функция переходов НКА имеет вид $\delta : Q \times \Sigma \to 2^Q$ (возвращает подмножество состояний). НКА допускает цепочку, если существует *хотя бы один* путь переходов, приводящий в допускающее состояние.

3. **Автоматы с ϵ-переходами (ϵ-НКА).** Это расширение НКА, где автомату разрешено спонтанно менять состояние, не читая входной символ (по пустой цепочке ϵ). Для работы с ними вводится понятие ϵ-замыкания (ECLOSE) — множества всех состояний, достижимых из данного только по ϵ-переходам.

В § 2.3.5 (стр. 77–80) доказывается важнейшая теорема: *любой язык, допускаемый НКА (или ϵ-НКА), может быть допущен и некоторым ДКА*. Доказательство строится с помощью «конструкции подмножеств», где каждое состояние нового ДКА представляет собой множество состояний исходного НКА. Это доказывает, что недетерминизм не расширяет класса распознаваемых языков, но делает построение автоматов более удобным.

2. Эксперименты с автоматами и различимость состояний

[Взято из Главы 4 «Свойства регулярных языков», § 4.4 «Эквивалентность и минимизация автоматов», стр. 170–180]

В контексте теории автоматов под «экспериментами» часто понимается задача определения внутреннего состояния автомата или его эквивалентности эталону. В учебнике Хопкрофта эта тема раскрывается через призму **эквивалентности и различимости состояний**.

Два состояния p и q одного и того же ДКА называются **эквивалентными**, если для любой входной цепочки w автомат, начав с состояния p, придет в допускающее

состояние тогда и только тогда, когда он придет в допускающее состояние, начав с q. Если существует хотя бы одна цепочка w, различающая эти состояния, они называются **различимыми**.

Для проведения такого «эксперимента» (т.е. проверки различимости) используется **алгоритм заполнения таблицы** (стр. 171–176):

1. Строится таблица, по осям которой перечислены все пары состояний автомата.
2. В ячейки, где одно состояние допускающее, а другое — нет, ставится крестик (эти состояния очевидно различимы на пустой цепочке ϵ).
3. Алгоритм итеративно просматривает оставшиеся пустые ячейки. Если при переходе по некоторому символу a пара состояний $\{p, q\}$ переходит в пару $\{p', q'\}$, которая уже помечена крестиком, то и исходная пара $\{p, q\}$ помечается как различимая.

Если две ячейки остаются неотмеченными до конца работы алгоритма, состояния эквивалентны. Данный метод позволяет не только проводить «эксперимент» по распознаванию поведения автомата, но и лежит в основе **минимизации ДКА** — построения автомата с наименьшим числом состояний, эквивалентного исходному.

3. Алгебра регулярных выражений

[Взято из Главы 3 «Регулярные выражения и языки», § 3.4 «Алгебраические законы для регулярных выражений», стр. 132–139]

Регулярные выражения представляют собой алгебраический (декларативный) способ задания регулярных языков. Они строятся из базовых элементов (символы алфавита, пустая цепочка ϵ, пустое множество $\emptyset$) с помощью трех операций:

- **Объединение** ($L + M$ или $L \cup M$);
- **Конкатенация** (LM или $L \cdot M$);
- **Итерация (замыкание Клини)** (L^*).

Множество регулярных выражений образует алгебру, подчиняющуюся ряду законов (стр. 132–138):

1. **Ассоциативность:** $L + (M + N) = (L + M) + N$ и $L(MN) = (LM)N$.
2. **Коммутативность объединения:** $L + M = M + L$ (конкатенация некоммутативна!).
3. **Дистрибутивность:** $L(M + N) = LM + LN$ и $(M + N)L = ML + NL$.
4. **Идемпотентность объединения:** $L + L = L$.
5. **Единичные и нулевые элементы:**
   - $\emptyset + L = L + \emptyset = L$ ($\emptyset$ — единица для объединения);
   - $\epsilon L = L\epsilon = L$ (ϵ — единица для конкатенации);
   - $\emptyset L = L\emptyset = \emptyset$ ($\emptyset$ — аннулятор для конкатенации).

6. **Законы итерации:**

   - $(L^*)^* = L^*$;
   - $\emptyset^* = \epsilon$;
   - $L^+ = LL^*$ (где L^+ — это одна или более итераций);
   - $L^* = L^+ + \epsilon$.

Знание этих законов позволяет упрощать сложные регулярные выражения, не прибегая к построению автоматов. Проверка истинности алгебраических законов сводится к подстановке вместо переменных конкретных символов и проверке равенства получаемых языков.

4. Теорема Клини о регулярных языках

[Взято из Главы 3 «Регулярные выражения и языки», § 3.2 «Конечные автоматы и регулярные выражения», стр. 108–125]

Фундаментальный результат, связывающий «машинный» (автоматы) и «алгебраический» (регулярные выражения) подходы, известен как **Теорема Клини**.

Формулировка теоремы: Класс языков, допускаемых конечными автоматами (ДКА, НКА или ϵ-НКА), в точности совпадает с классом языков, задаваемых регулярными выражениями.

Доказательство теоремы состоит из двух частей (направлений):

1. **От автомата к регулярному выражению** (§ 3.2.1–3.2.2, стр. 109–120). Доказывается, что для любого ДКА можно построить регулярное выражение, задающее его язык. В учебнике это делается с помощью метода **исключения состояний**. Вводятся обозначения $R_{ij}^{(k)}$ — регулярное выражение для путей из состояния i в состояние j, проходящих только через состояния с номерами не больше k. Процесс начинается с $k = 0$ (прямые переходы) и итеративно наращивается до $k = n$ (все состояния).

2. **От регулярного выражения к автомату** (§ 3.2.3, стр. 120–125). Доказывается с помощью **структурной индукции** по построению регулярного выражения. Базовые случаи (символы, ϵ, $\emptyset$) реализуются простейшими автоматами. Затем показывается, как из автоматов для выражений R и S сконструировать автоматы для $R + S$, RS и R^* с помощью добавления новых начальных/допускающих состояний и ϵ-переходов (так называемая конструкция Томпсона).

Таким образом, теорема Клини обосновывает эквивалентность трех моделей: ДКА, НКА и регулярных выражений. Это означает, что для описания регулярных языков можно свободно выбирать наиболее удобный инструмент: автомат (для программирования лексических анализаторов) или регулярное выражение (для компактной записи шаблонов поиска).